UNIVERSITY OF DA NANG
UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF PROJECT MANAGEMENT

GRADUATION THESIS

DEPARTMENT: INDUSTRIAL ENGINEERING AND MANAGEMEMT

DEVELOPING A SMART FACTORY
A CASE OF CERAMIC TILE MANUFACTURING

Instructor : **DR. NHAT-TO HUYNH**
Student : **LE-VI-NHAN-TAM TRAN**
Student ID : **118180125**
Class : **18QLCN**

Da Nang, 12/2022

REVIEWER COMMENTS TO AUTHOR(S):

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

SUMMARY

1. Graduation thesis title:
Developing a smart factory: a case of ceramic tile manufacturing.
2. Student: Le Vi Nhan Tam TRAN
3. Student ID: 118180125 4. Class: 18QLCN
5. Summary

In general, this thesis attempts to suggest a framework for a "smart factory" that is built on data collection and machine learning algorithms to adapt the Industry 4.0 with IIoT and digital transformation, particularly in the case of ceramic manufacturing.

So, I developed "AdUsin" to prove the rightness of this framework. "AdUsin" is a web application using Python WebFramework - Streamlit which has 3 main modules:
- Database Module for IoT device data storage, database operation and data display.
- Operation Management Module includes Aggregate Planning, Predictive Maintenance and Quality Control (QC). Additionally, the QC is a group of assistance tasks:

+ Control chart: recognize and notify out of control point, real-time control of the object's attributes.

+ Machine status: tracked the status of them through display monitoring.

+ Material Defect Detection: The YOLOv7 model's first version was used to identify and categorize typical ceramic failure using imported photos or real-time camera input, but I've tried to apply version 2 with the GACNN - Genetic Algorithm to optimize the model's CNN structure.

- Business - Finance Module currently used Linear Regression for the Forecasting Demand function and I'm working to improve the Payroll function.

UNIVERSITY OF DA NANG
UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF PROJECT MANAGEMENT

SOCIALIST REPUBLIC OF VIETNAM
Independence-Freedom-Happiness

GRADUATION THESIS MISSIONS

Full Name: Le Vi Nhan Tam TRAN Student ID: 118180125 Class: 18QLCN
Faculty: Project Management Department: Industrial Engineering and Management

1. *Graduation thesis title:*

- Developing a smart factory: a case of ceramic tile manufacturing.

2. *Topics Covered:* ☐ S*igned an intellectual property agreement for the implementation*
3. *Contents of narrations and formulations:*

- Chapter 1: An introduction to the broad range of issues, such as waste and the potential utility of data. Some production flaws in ceramics, particularly in quality control.
- Chapter 2: Review of research articles related to IIoT, database administration, smart factory framework, ceramic failure, machine learning, and image processing utilizing the YOLO v7 and GACNN algorithms.
- Chapter 3: Present the suggestion framework, the specification of the application architecture, the algorithm and the functional illustration.

4. *Technical drawing, software (details requirement if required):*

- Flowchart & diagram: Presented are the suggested framework's architecture, use case system description, functional demonstration and machine learning algorithms: classification, regression, genetic, YOLO, convolutional neural network.
- Software: MySQL Database Management: phpMyAdmin + XAMPP; API platform: Postman; IDE Compiler: Pycharm + Arduino; Label : LabelMe + Roboflow; Programming Languages: Python (main) + PHP (database) + C++ (microcontrollers).

5. *Instructor:* Dr. Nhat To HUYNH
6. *Thesis assignment date: 22/08/2022*
7. *Thesis complete date: 12/12/2022*

Da Nang, 12th December, 2022

Head of IEM department **Instructor**

Dr. Nhat To HUYNH

ABSTRACT

Topic: A suggestion of Smart Factory framework with applying Machine Learning techniques in the case study of ceramic tiles industrial.

Authors: Dr. Nhat To HUYNH and Le Vi Nhan Tam TRAN.

Purpose: The ambition of this project is to provide a smart factory framework based on the idea presented through this documentation and the demo of the smart factory application called 'AdUsin'. AdUsin is a web application using the Streamlit framework and has modules to solve the industry-specific process in operation management and data collection. By applying Artificial Intelligence algorithms to improve productivity, accuracy,... for adapted the requirement of industrial.

Method: based on the IoT devices to collect data of object's attribute then storage into a Relational Database Management System (RDBMS). While solving the industry-specific process by Machine Learning algorithms, those data would be pre-processed to be the input of training models, which result would be used in the application.

Challenge: Due to the reality of the industry, this project may be faced many challenges such as data, compatibility, information security, scalability, network externalities, ... and environmental which mainly affect sustainable development.

Keywords: Smart factory application; Ceramic tiles failure; YOLOv7, GACNN; Defect detection and classification; Operation management.

Feedback is a crucial component of any academic writing process, so I would like to thank the reviewer and look forward to the insightful comments to help me increase my knowledge and avoid making the same mistakes again. Last but not least, a big thanks go to Mr. To who suggested this topic and instructed me to complete the project. I also want to thank my brothers and my friends for sharing their time, enthusiasm and knowledge to support me through this duration.

GUARANTEE

This thesis was written by me during the fall semester of 2022 as the final degree project within the graduate program "Industrial Engineering and Management" at the University of Da Nang - University of Science and Technology, Vietnam.

During the process to complete this thesis project, with factual ignorance thus, the inevitable gap is a sure thing. But I guarantee that I did not use any cheating methods in completing this thesis. While in progress, the official resources and references were used.

I, Le Vi Nhan Tam TRAN, am totally responsible if any issues arise.

Danang, 12th December, 2022

CONTENTS

LIST OF FIGURES....1

LIST OF TABLES....2

LIST OF ABBREVIATIONS....2

CHAPTER 1: INTRODUCTION....3

CHAPTER 2: LITERATURE REVIEW....4

CHAPTER 3: SMART FACTORY APPLICATION....6

3.1. Database Module....8

3.1.1 Create Database....8

3.1.2 Data Collection....9

3.1.3 Database Operation....11

3.2 Operation Management Module - Main....12

3.2.1 Control Chart....12

3.2.2 Monitoring....13

3.2.3 Material Defect Detection....14

3.3 Operation Management Module - Beta....22

3.3.1 Aggregate Planning....22

3.3.2 Predictive Maintenance....23

3.4 Business Module....24

3.4.1 Demand Forecasting....24

3.4.2 Payroll....25

REFERENCES....26

LIST OF FIGURES

Figure 1: Suggestion of smart factory framework 3
Figure 2: The Model – View - Controller Pattern 6
Figure 3: Web App User Interface 7
Figure 4: Use Case Diagram 7
Figure 5: phpMyAdmin User Interface 8
Figure 6: Data Collection 9
Figure 7: IoT manufacturing Architecture 10
Figure 8: Data Collection Progress 10
Figure 9: CRUD Operations 11
Figure 10: CREATE Operation 11
Figure 11: Control Chart Function 12
Figure 12: ON / OFF User Interface 13
Figure 13: Monitoring's architecture 13
Figure 14: YOLO's architecture 14
Figure 15: YOLO's output 14
Figure 16: Ceramic Defects Detection Progress 15
Figure 17: UI detection result by YOLOv7 model 16
Figure 18: Genetic Algorithm Progress 16
Figure 19: GACNN Framework 17
Figure 20: Common defect types of ceramic tiles surface 18
Figure 21: Example of fixed CNN structure 19
Figure 22: Example of chromosome 21
Figure 23: UI detection result by GACNN model 21
Figure 24: Aggregate Planning UI 22
Figure 25: Predictive Maintenance UI 23
Figure 26: Demand Forecasting UI 24
Figure 27: Payroll's architecture 25
Figure 28: Payroll user interface 25

LIST OF TABLES

Table 1: Python Frameworks Benchmark.....6
Table 2: Details description of ceramic's defect 18
Table 3: Details description of activation functions 20
Table 4: Sample data of aggregate planning.....22
Table 5: Sample data of predictive maintenance 23
Table 6: Benchmark of PM's models 23
Table 7: Sample data of demand forecasting.....24

LIST OF ABBREVIATIONS

DB	: Database
RDBMS	: Relational Database Management System
UX	: User Experience
UI	: User Interface
AI	: Artificial Intelligence
ML	: Machine Learning
IoT	: Internet of things
IIoT	: Industrial Internet of Things
CNN	: Convolutional Neural Networks
NN	: Neural Networks
GA	: Genetic Algorithm
GACNN	: Genetic Algorithm Convolutional Neural Networks
YOLO	: You only look one
CRUD	: Create – Read – Update - Delete
AR	: Augmented Reality
VR	: Virtual Reality
M2M	: machine-to-machine

CHAPTER 1: INTRODUCTION

In this day and age, we are living with the outburst of technology, without these trends, ***industrial production*** must improve to adapt to the demands. Especially in the digital era, ***data*** is focused on because of the value they bring back. Due to the situation industrial, which creates millions of kinds of data day by day but we don't collect it, we waste it which is a big mistake.

Besides that, one of the few sectors in Vietnam that has advanced to the regional and global levels is the ***ceramic tile*** industry, on which I concentrate on the project. As of right moment, the total capacity is 500 million m2/year, placing it first in Southeast Asia and sixth overall. Granite production bricks in particular have a small capacity now, but by 2021 it would be over 60 million m2/year. However, the level of competition in the ceramic tile business would increase significantly over the next several years, particularly in the ceramic tile category. In addition, the ceramic tile industry bulk amount of ceramic tiles are manufactured, it is very difficult to monitor the ***quality*** of each and every tiles manually. Lot of human resources are required for the ***defect detection*** of the tiles. Also it is quite tedious and time consuming method.

So, for ***sustainable development***, digital transformation must be taken care of by the enterprise, with aim to be a smart factory for industry 4.0. And the key is data, the enterprise should have planned in data collection for applying ***Artificial Intelligence algorithms*** in future development.

In the figure 1 below: I suggestion a framework which based on the combination of IoT devices, Relational Database Management System (RDBMS), Streamlit framework and Machine Learning algorithms.

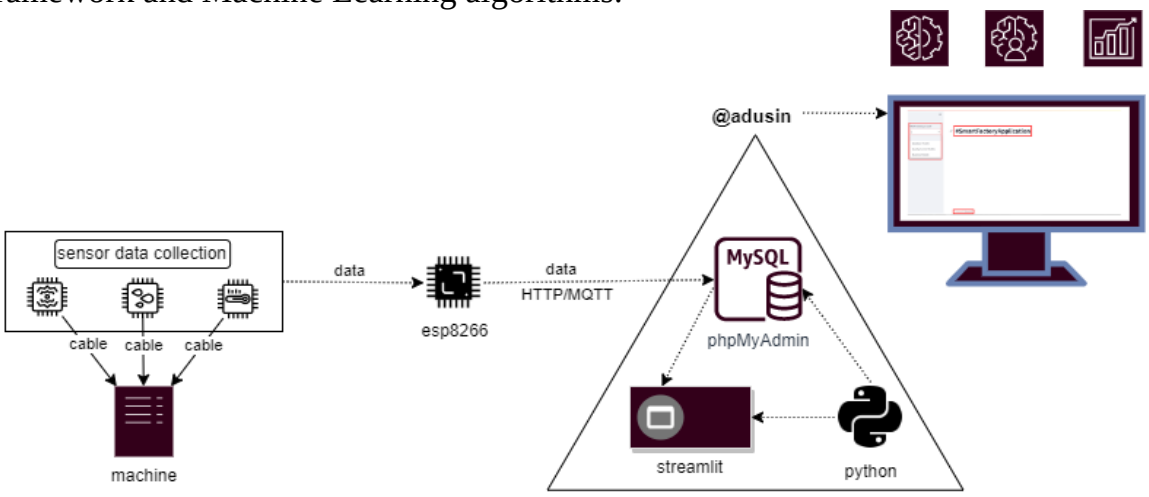


Figure 1: Suggestion of smart factory framework

CHAPTER 2: LITERATURE REVIEW

Day by day, IoT was being a common term that stands for the things that are connected by the Internet but moreover, ***Shariatzadeh, N., Lundholm, T., Lindberg, L., & Sivard, G. (2016) in their paper: "Integration of digital factory with smart factory based on Internet of Things"*** has mentioned that IoT not only deals with smart connections between physical objects but also with the interaction with different IT tools used within the digital factory, the IoT in manufacturing can be defined as a future where everyday physical objects on the shop floor, people and systems (things) are connected by the Internet to build services critical to the manufacturing. As for, digitalization has improved the production scene and IoT in manufacturing, also known as the Industrial Internet of Things (IIoT), is a step toward transforming conventional factories into smart places. Which has a wide range of applications, including the ability to anticipate problems before they occur, increase profits and customer satisfaction, improve asset management through a connected supply chain, implement operational intelligence, increase quality improve production planning and scheduling, etc. According to the article ***"Forensic Analysis on Internet of Things (IoT) Device Using Machine-to-Machine (M2M) Framework" by Mazhar, M. S., Saleem, Y., Almogren, A., Arshad, J., Jaffery, M. H., Rehman & Hamam, H***, in 2022, employee safety, security, and privacy, machine-to-machine (M2M) automation and IoT blended with VR and AR bringing digital and physical worlds closer. The article ***"IoT-based personal health care monitoring device for diabetic patients" by Rahman and his partners (2017)*** was using Arduino used to read the sensor with sense the breath then shared to the database via ESP8266 Wi-Fi Module and can be accessed by the patients or registered doctors which purpose the accuracy of the current microcontrollers. Recently, Raspberry Pi - microcontrollers have been introduced with the expectation to be the future of IoT.

In the ceramic tiles industry, ***Boukouvalas released the paper "Ceramic tile inspection for color and structural defects"*** with the methods including crack and pinhole detectors for plain tiles based on a set of separable line filters, crack detectors for textured tiles based on the Wigner distribution and a novel conjoint spatial-spatial frequency representation of texture, and a color texture tile defect detection algorithm that looks for anomalies in both the chromatic and structural properties of textured tiles.

On a variety of tiles with both fake and real faults, the aforementioned automatic inspection techniques have been put into practice and tested. The findings imply that the performance is sufficient to serve as a foundation for an effective commercial visual inspection system. Then ***Shire with "Plain Ceramic Tiles Surface Defect Detection Using Image Processing"*** had shown automated classification method helps us to monitor the defects within a very short period and also to decide about the recovery process so that the defective tiles may not be mixed with the good quality tiles.

In 2020, CNNs have achieved remarkable success in many modern image classification tasks. However, CNN's performance is highly architecture dependent. Modern CNN architectures are often manually designed using expertise in both the CNN and the problem under study. Therefore, it is difficult for users without extensive CNN experience to design the optimal CNN architecture for their own image classification problem of interest. The following research by ***Sun and its partners with the topic "Automatically Designing CNN Architectures Using the Genetic Algorithm for Image Classification"*** proposed a method of automatically designing his CNN architecture using genetic algorithms to effectively deal with the task of image classification. The main benefit of the offered method is its "automated" component, which enables users to utilize it without any prior knowledge of CNNs and yet build an appropriate CNN architecture for the given images. The experimental results show that the proposed method outperforms the currently employed automatic CNN architecture design strategies in terms of classification accuracy, parameter counts, and computational resource consumption. The suggested method displays classification accuracy that is very comparable to the best from manually produced, automatically created, and manually tuned CNNs while requiring fewer computational resources. Then in June 2022 ***Mantau and his partners*** released the article***: "A GA-Based Learning Strategy Applied to YOLOv5 for Human Object Detection in UAV Surveillance System"*** had proposed a new approach to using a GA within a YOLOv5 framework for human object detection applied in the Unmanned Aerial Vehicle (UAV) perspective image dataset. The dataset has challenges, such as a small target, the view of the object being from above, and there is an illumination and light effect. The end result is that when compared to the original YOLOv5 for Human Detection on Unmanned Aerial Vehicle Perspective, this YOLOv5-based transfer learning approach employing the RGB-TIR dataset and improved by GA may reach greater accuracy.

CHAPTER 3: SMART FACTORY APPLICATION

The Smart Factory application currently has the architecture shown through the MVC model below, including the following components:

+ Model: tables in databases (MySQL) would record information and data transmitted by the Device used to track the required parameters.

+ Controller: these data would be processed by Python and then used as input for models: machine learning or deep learning then the results would be used to support the decision-making process.

+ View: interactive visualization of data, features, UX, UI are shown through Streamlit - is an open-source Python Library, that makes it easy to create a web app for local testing, the demo of Machine Learning products

The extremely important step was choosing the software framework. Table 1 has the description of the alternatives I have thought about.

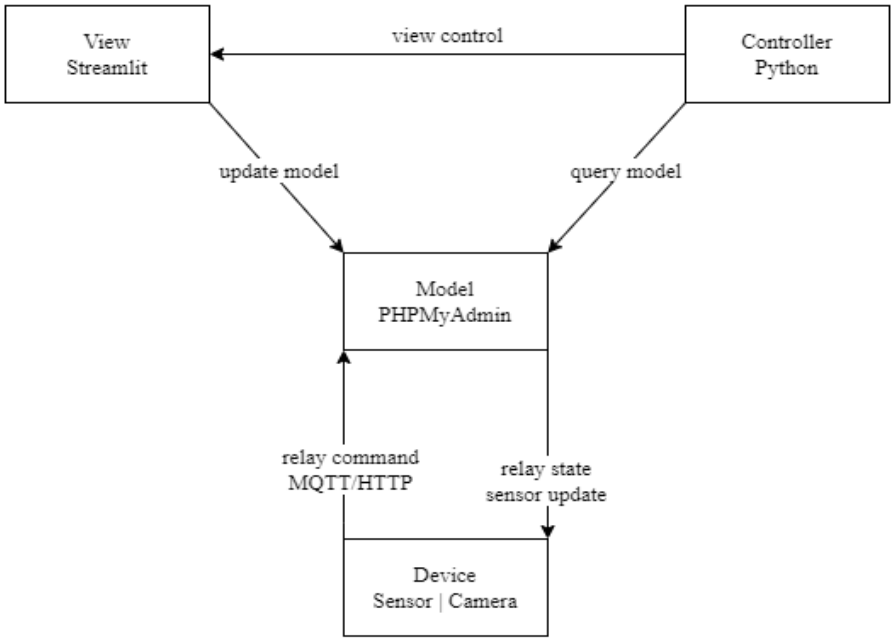


Figure 2: The Model – View - Controller Pattern

Table 1: Python Frameworks Benchmark

	Flask	**Django**	**Dash**	**Streamlit**
First released	2010	2005	2017	2019
Simplicity	~ django	~flask	< flask	< flask
Focus	web framework	web framework	dashboard	dashboard
Adaptability	~ django	~ flask	< flask	< dash

There are 3 modules in this demo included: Database Module, Operation Management Module and Business – Finance Module.



Figure 3: Web App User Interface

Based on the system specification diagram below, the system is divided into modules with functions serving each department. Decentralized interaction and identification according to the user's department.

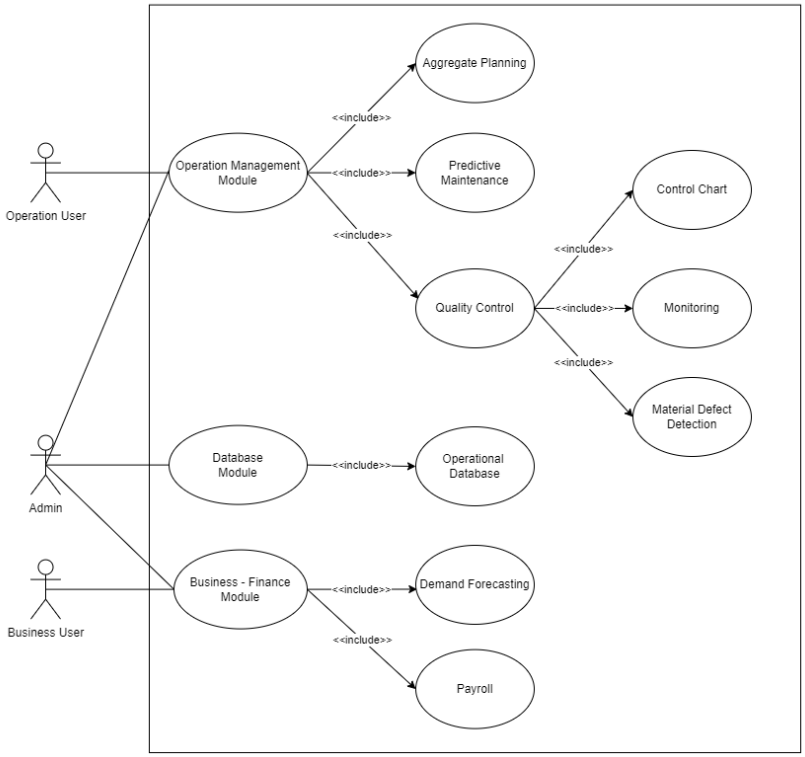


Figure 4: Use Case Diagram

3.1. Database Module

MySQL is an open-source database management system (referred to as RDBMS for short) that operates under the client-server model, has high speed, stability and is quite easy to use in several operating systems.

A database is a collection of data with the same structure placed in a common dataset. The database is organized and arranged like a closely linked spreadsheet.

3.1.1 Create Database

phpMyAdmin is a web application that provides a GUI in conjunction with the MySQL database management system. Used to set up, manage the database system and execute SQL queries.

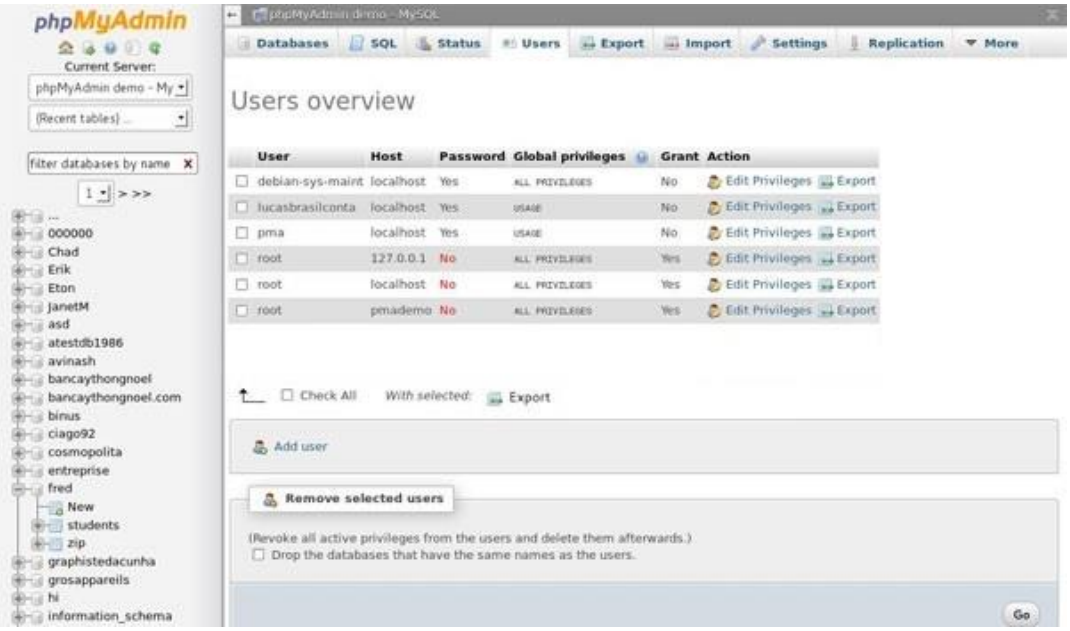


Figure 5: phpMyAdmin User Interface

phpMyAdmin has almost all the features of MySQL (create, copy, drop, rename and change databases, tables, fields and indexes, manage MySQL user accounts and privileges,...) export data in multiple formats, manage multiple servers, monitor databases and so on.

The database would be set up for each object and collects the characteristics of that object, for example, the Dataset – Machines database would include Machine A, Machine B,… and Database Machine A would store the characteristics such as application. capacity, temperature, torque and timestamp, then identified by ID following the table structure.

3.1.2 Data Collection

Data with types like numbers, characters and strings such as temperature, capacitance, optics, humidity, hall, pressure, ... would be monitored and collected by specialized sensors (Device). After that, it is sent to the server via MQTT protocol, etc. via the ESP8266 board (or embedded computer so that it can connect to the Internet and communicate with the server via MQTT protocol). For images, it would be collected through cameras and then processed for machine learning models.

The device can also be a monitor, a computer with the function of displaying information in the form of dashboards, announcements, warnings, reports, ...are updated in real-time to support the decision-making process.

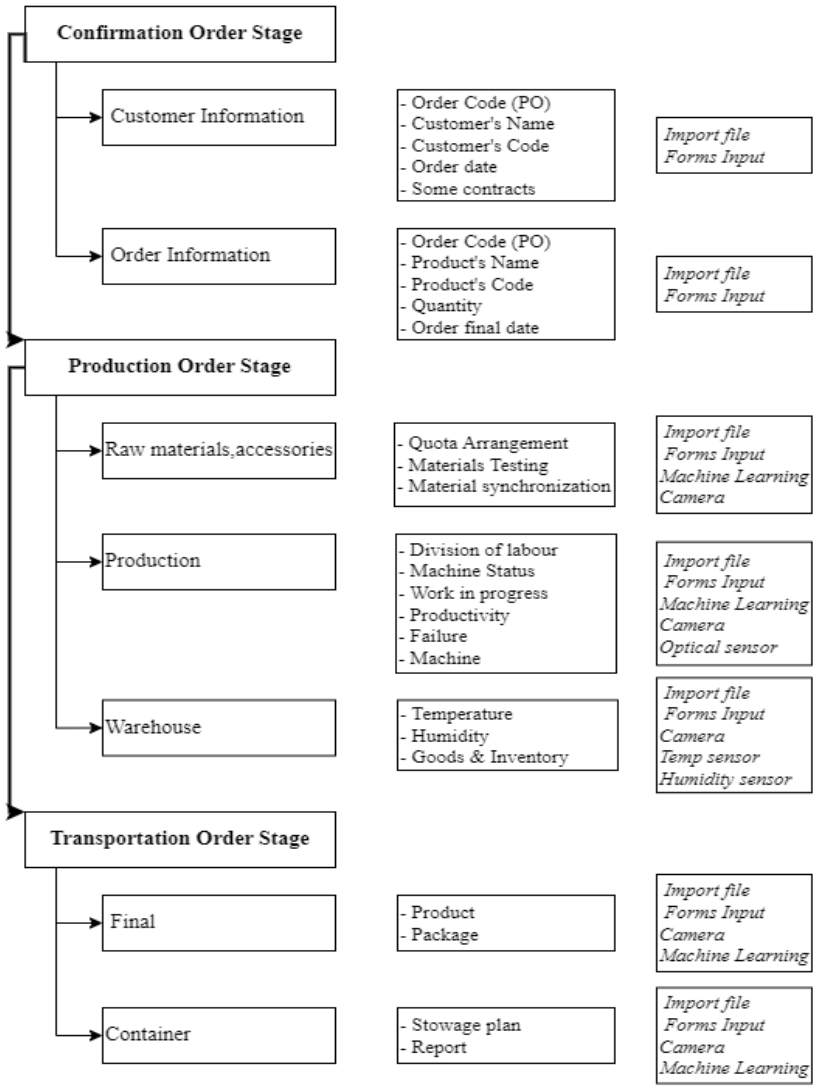


Figure 6: Data Collection

Platform for internet of things (IoT) have some basic requirements: secure, reliable and cheap. Which could access to our system from everywhere with different connection options.

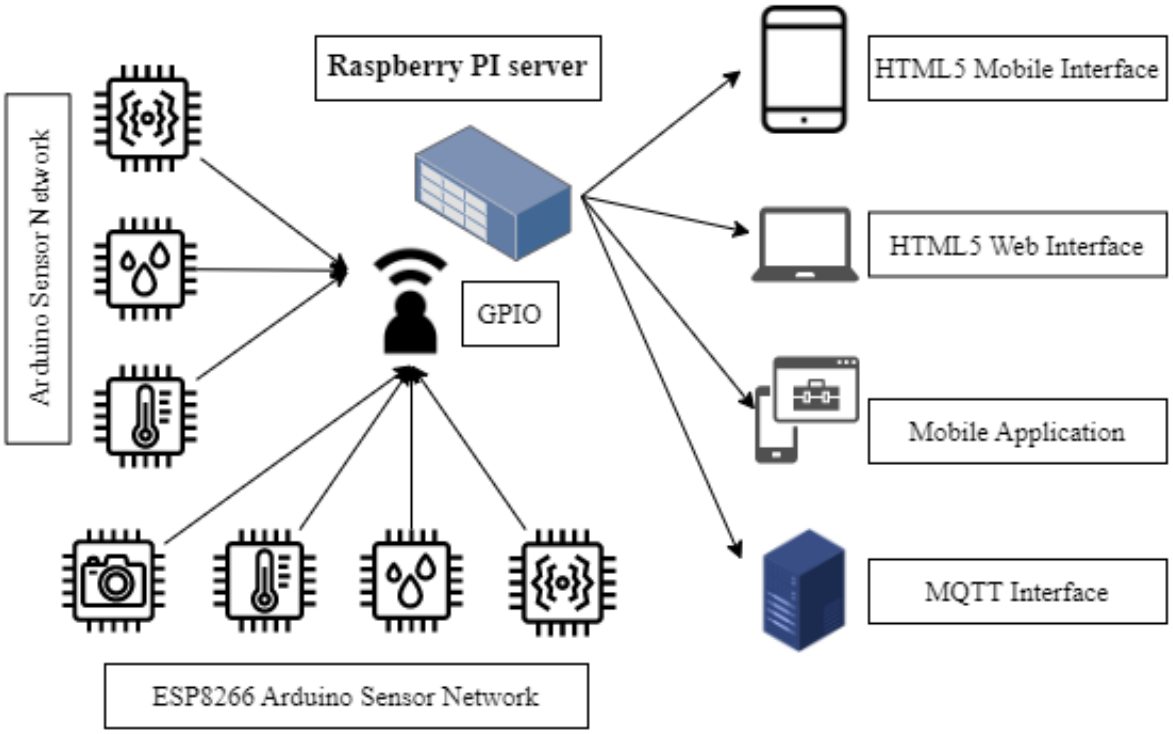


Figure 7: IoT manufacturing Architecture

The ESP8266 is a chip in the QFN package that is capable of both a TCP/IP suite and a microcontroller. The ESP8266 provides a highly integrated WiFi solution that meets the needs of IoT applications. ESP8266 NodeMCU contains the important components of a computer: CPU, RAM, network (WiFi) and modern SDK. When the sensors connect to the EPS8266, the data which has been collected by sensors would be sent to the ESP8266 and then sent to the database after each specified cycle (about 3s ~ depends a lot on the transmission speed of the network).

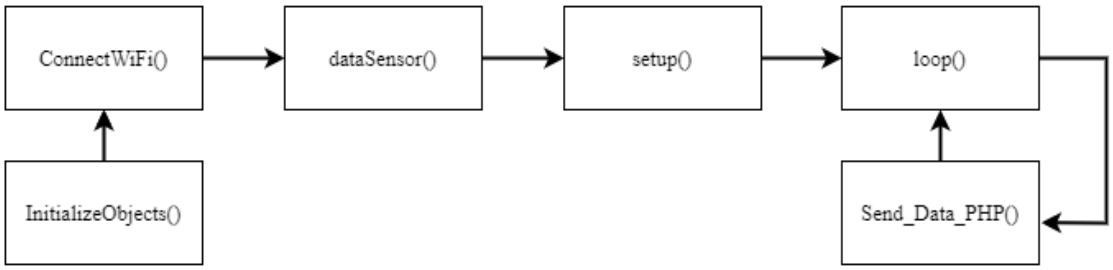
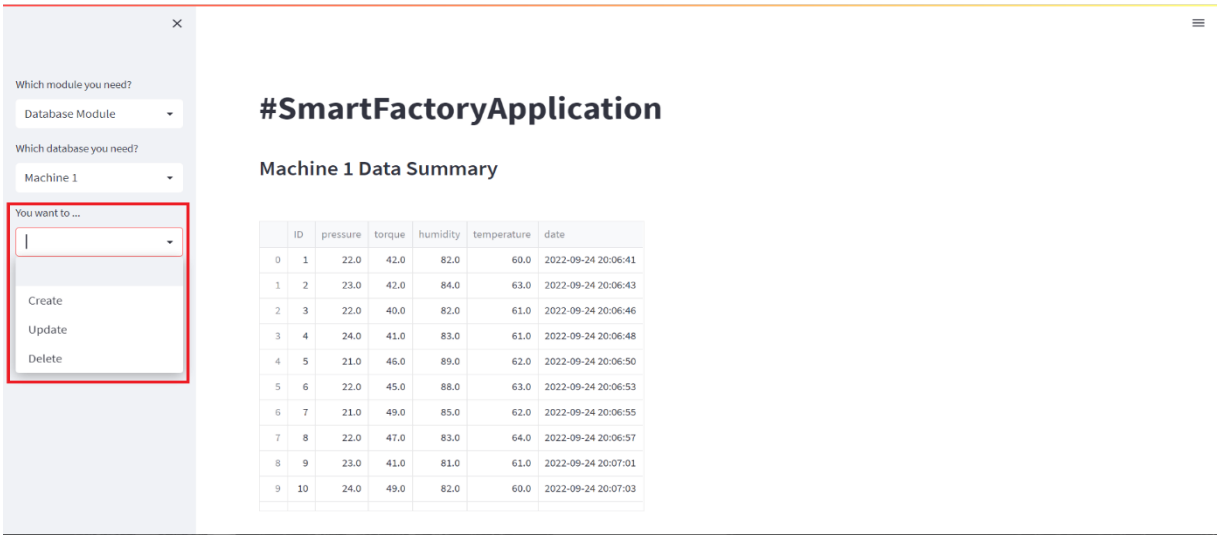


Figure 8: Data Collection Progress

3.1.3 Database Operation

Data Summary allows to read and display of all data that have been sent by sensors to the database of the object user.

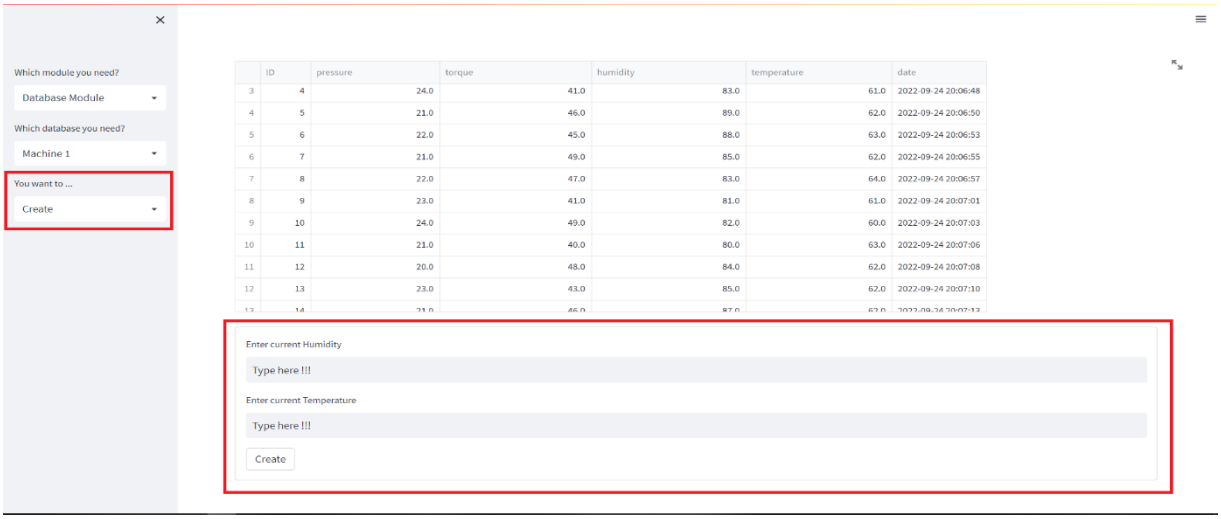


	ID	pressure	torque	humidity	temperature	date
0	1	22.0	42.0	82.0	60.0	2022-09-24 20:06:41
1	2	23.0	42.0	84.0	63.0	2022-09-24 20:06:43
2	3	22.0	40.0	82.0	61.0	2022-09-24 20:06:46
3	4	24.0	41.0	83.0	61.0	2022-09-24 20:06:48
4	5	21.0	46.0	89.0	62.0	2022-09-24 20:06:50
5	6	22.0	45.0	88.0	63.0	2022-09-24 20:06:53
6	7	21.0	49.0	85.0	62.0	2022-09-24 20:06:55
7	8	22.0	47.0	83.0	64.0	2022-09-24 20:06:57
8	9	23.0	41.0	81.0	61.0	2022-09-24 20:07:01
9	10	24.0	49.0	82.0	60.0	2022-09-24 20:07:03

Figure 9: CRUD Operations

Besides, authorized users to interact with the database could:

+ Create: allows to import - create a new data point to write to the database (the timestamp happens at the time of selecting the "Create" button).



	ID	pressure	torque	humidity	temperature	date
3	4	24.0	41.0	83.0	61.0	2022-09-24 20:06:48
4	5	21.0	46.0	89.0	62.0	2022-09-24 20:06:50
5	6	22.0	45.0	88.0	63.0	2022-09-24 20:06:53
6	7	21.0	49.0	85.0	62.0	2022-09-24 20:06:55
7	8	22.0	47.0	83.0	64.0	2022-09-24 20:06:57
8	9	23.0	41.0	81.0	61.0	2022-09-24 20:07:01
9	10	24.0	49.0	82.0	60.0	2022-09-24 20:07:03
10	11	21.0	40.0	80.0	63.0	2022-09-24 20:07:06
11	12	20.0	48.0	84.0	62.0	2022-09-24 20:07:08
12	13	23.0	43.0	85.0	62.0	2022-09-24 20:07:10

Figure 10: CREATE Operation

+ Update: allows selecting and editing (updating) a data point in the database (only changing the value, not changing the time).

+ Delete: allows to selection and deletes a data point in the database (the ID of the data point would be deleted accordingly).

Due to the time constraints of my thesis project, I was unable to release all of the features in the Operation Management module. As a result, I divided the documentation into 2 sections, the first one is tha *Main* standard for functions that were released, like Quality Control, and the last one is *Beta* for functions that were demoed, including Aggregate Planning and Predictive Maintenance.

3.2 Operation Management Module - Main

The Quality Control is a collection of support functions:

+ Control chart: real-time monitor control of the object's attribute, assisting in identifying and alerting to object anomalies (out of control point).

+ Monitoring: currently utilized to track machine status that could turn off the machine based on a range defined by the control chart, furthermore we remote control the status by the user interface.

+ Material Defect Detection: in the first version YOLOv7 model is applied to identify and classify common ceramic defects based on imported images or real-time camera and with similar input but I have tried to apply version 2 with GACNN - Genetic algorithm to optimize the CNNs structure of the model.

3.2.1 Control Chart

Use the data sent by EPS8266 to represent a control chart to support real-time monitoring of the observed object.

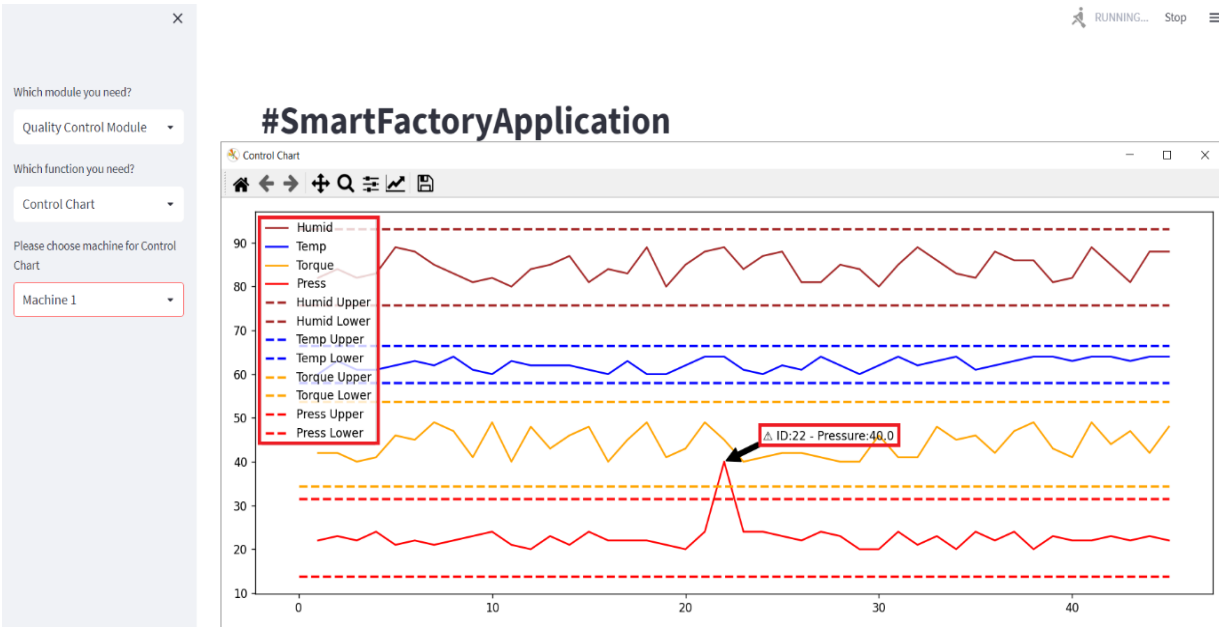


Figure 11: Control Chart Function

The Center Line equals either the average or median of the data .

$UCL = \bar{x} + m\,\partial/\sqrt{n}$

$LCL = \bar{x} - m\,\partial/\sqrt{n}$

n is the sample size

∂ is the estimated standard deviation of x

m is a multiplier (usually set to 3)

Based on UCL & LCL to identify and send alarms with out-of-control data points (Out of control) obtained from sensors to support the decision-making process to take corrective and control actions.

3.2.2 Monitoring

With modes, automatic on|off when an out-of-control event occurs and an on|off button in the user interface for reactive machine shutdown in response to circumstances.

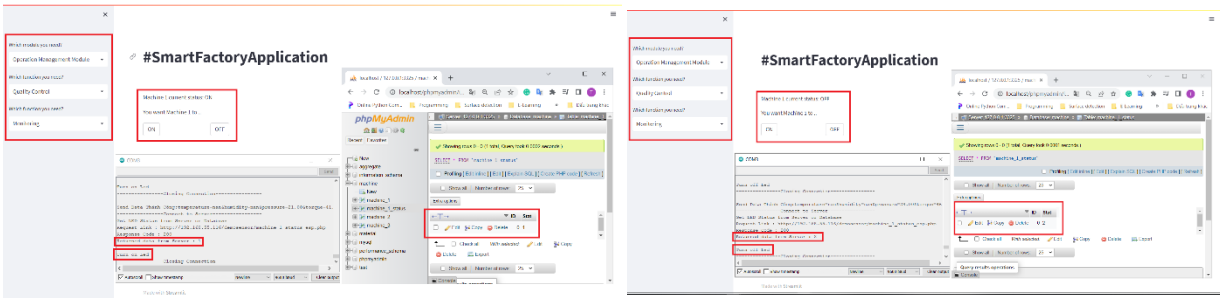


Figure 12: ON / OFF User Interface

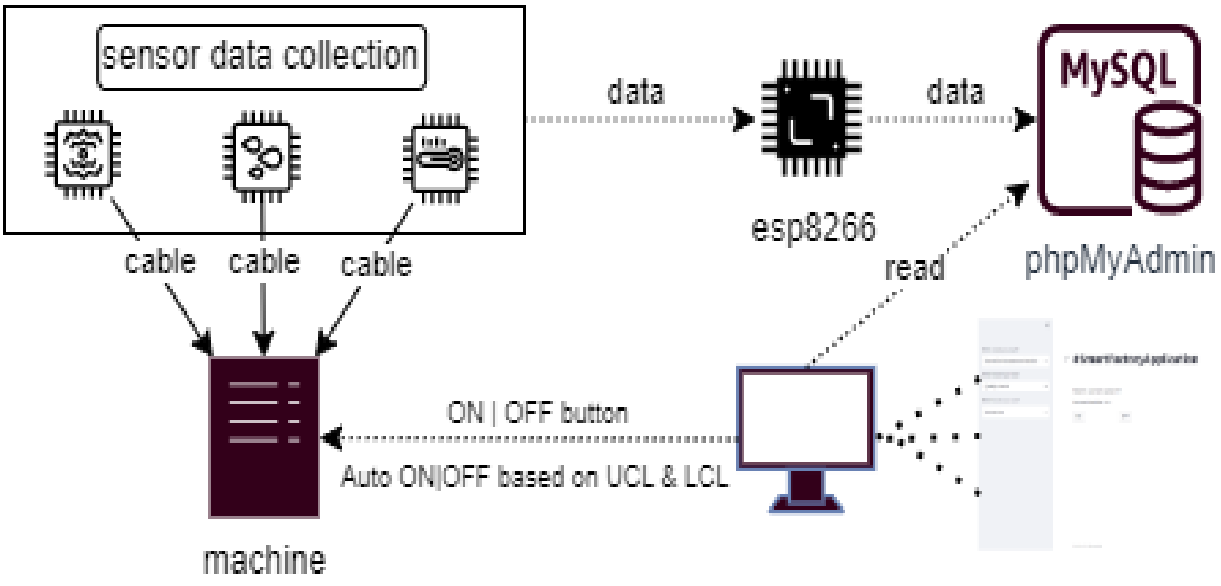


Figure 13: Monitoring's architecture

3.2.3 Material Defect Detection

After analyzing and collecting the dataset about product defects, I need to assign labels then divide into the train and the test dataset. With 7 classes included: blob, crack, scratch, eclipse, edge, pinhole and defect-free.

To solve this case, first let's take a look at YOLO - which stands for "You Only Look Once". It's a modern deep-learning algorithm commonly used for Object Detection purposes. The YOLO algorithm was trained on a specific type of data set consisting of 80 different types of classes. The YOLO architecture includes: base networks are convolution networks that perform feature extraction. The back part is the Extra Layers applied to detect objects on the feature map of the base network. YOLO's base network uses mainly convolutional layers and fully connected layers. YOLO architectures are also quite diverse and can be customized into versions for many different input shapes.

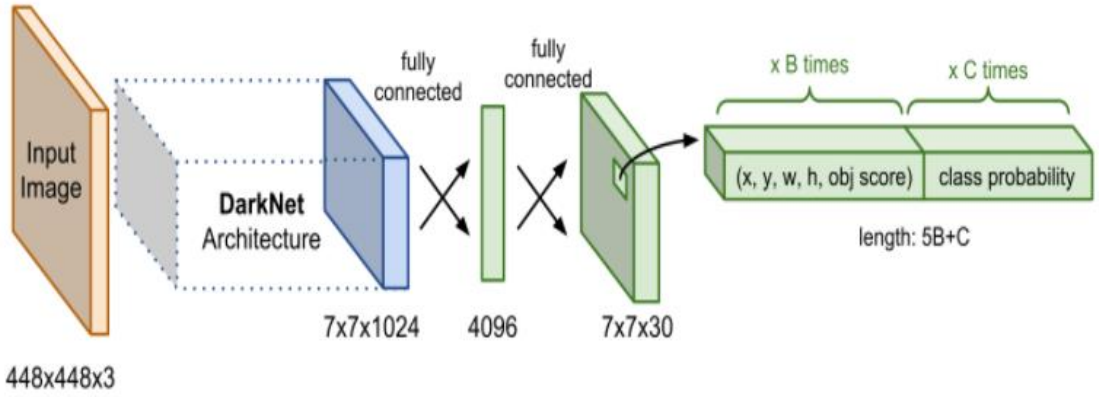


Figure 14: YOLO's architecture

YOLO outputs include:

+ P_0 is the predicted probability of the object appearing in the bounding box

+ (t_x, t_y, t_w,t_h) helps define the bounding box. Where t_x, t_y are the coordinates of the center and t_w, t_h are the width and length of the bounding box.

(P1, P1,…Pc) is the predictive probability distribution vector of classes

$$y^T = [p_0, \underbrace{\langle t_x, t_y, t_w, t_h \rangle}_{\text{bounding box}}, \underbrace{\langle p_1, p_2, \ldots, p_c \rangle}_{\text{scores of c classes}}]$$

Figure 15: YOLO's output

Starting from loading the YOLOv7 model, I use the training set to train. Through processing, the features of the learned object are extracted and saved into the features of the labels. After the evaluation & re-training process, the file weight.pt will be obtained.

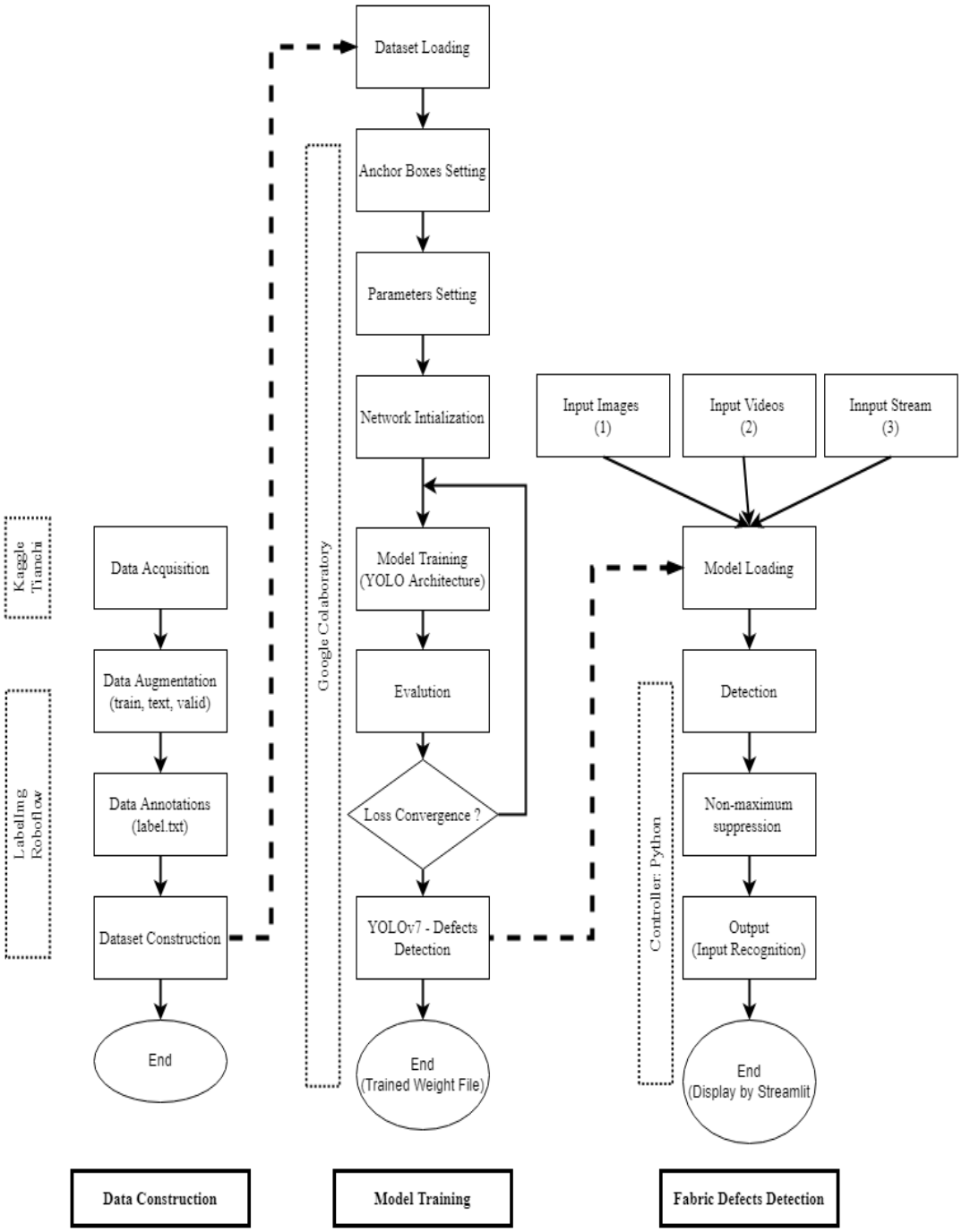


Figure 16: Ceramic Defects Detection Progress

After extracting the features of the video stream, the imported image or video, the model would compare with the features in the labels of the trained model, thereby determining the defect location and the defect name as shown in Figure 11.

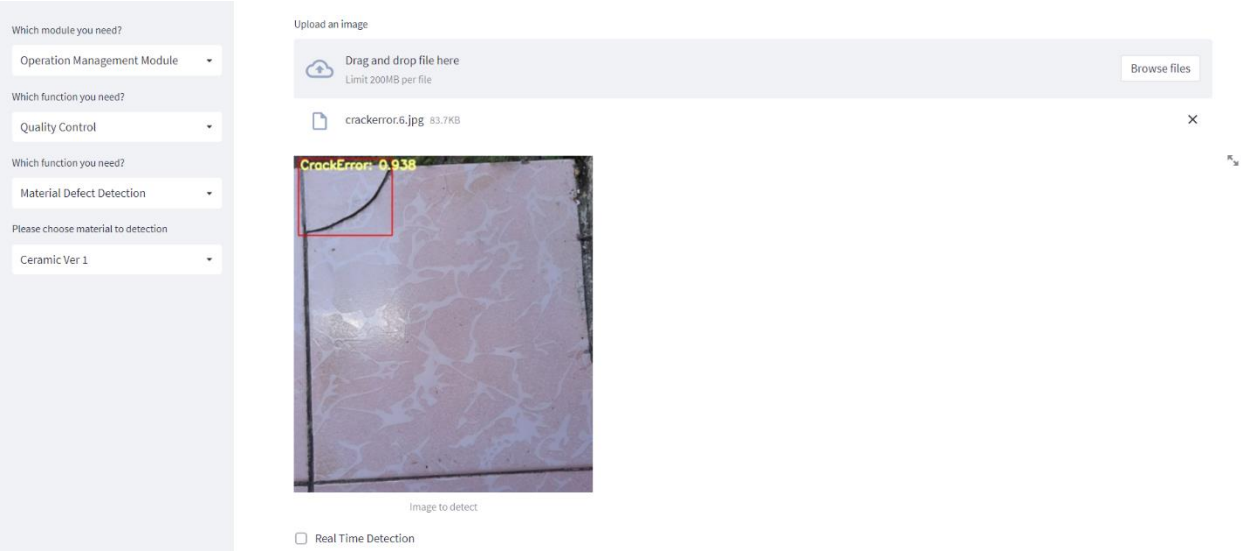


Figure 17: UI detection result by YOLOv7 model

GA - Genetic Algorithm is a search heuristic that is inspired by Charles Darwin's theory of natural evolution. The process have five phases:

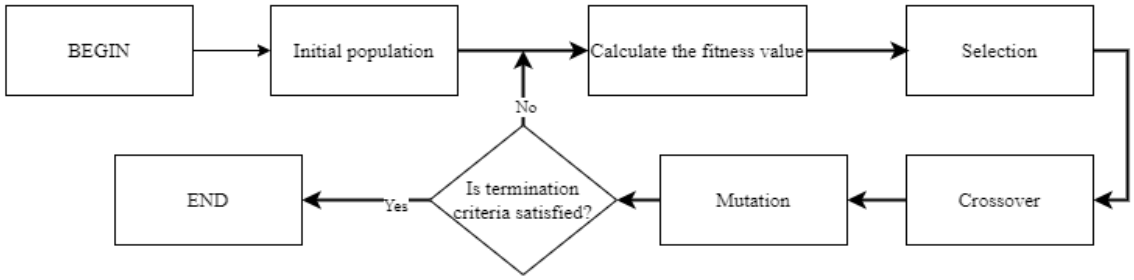


Figure 18: Genetic Algorithm Progress

To optimize CNN structure by GA, the first step is to create an initial population using an encoding routine. Every set of CNN structure parameters is represented by a chromosome. The processes of crossover and mutation are used to produce new offspring. Parents and children make up the transitory population that is utilized to assess and choose the next generation. Each chromosome's values in the test population are decoded and allocated to the CNN model. Based on a predetermined training dataset, CNN models are improved.

The best CNN models are then assessed using the test dataset to produce fitness values for each chromosome as shown in fitness function (weights for [P, R, mAP@0.5, mAP@0.5:0.95]). A selection procedure is used to choose the chromosomes for the following generation based on fitness ratings. Up until the termination condition is satisfied, this process is iterated.

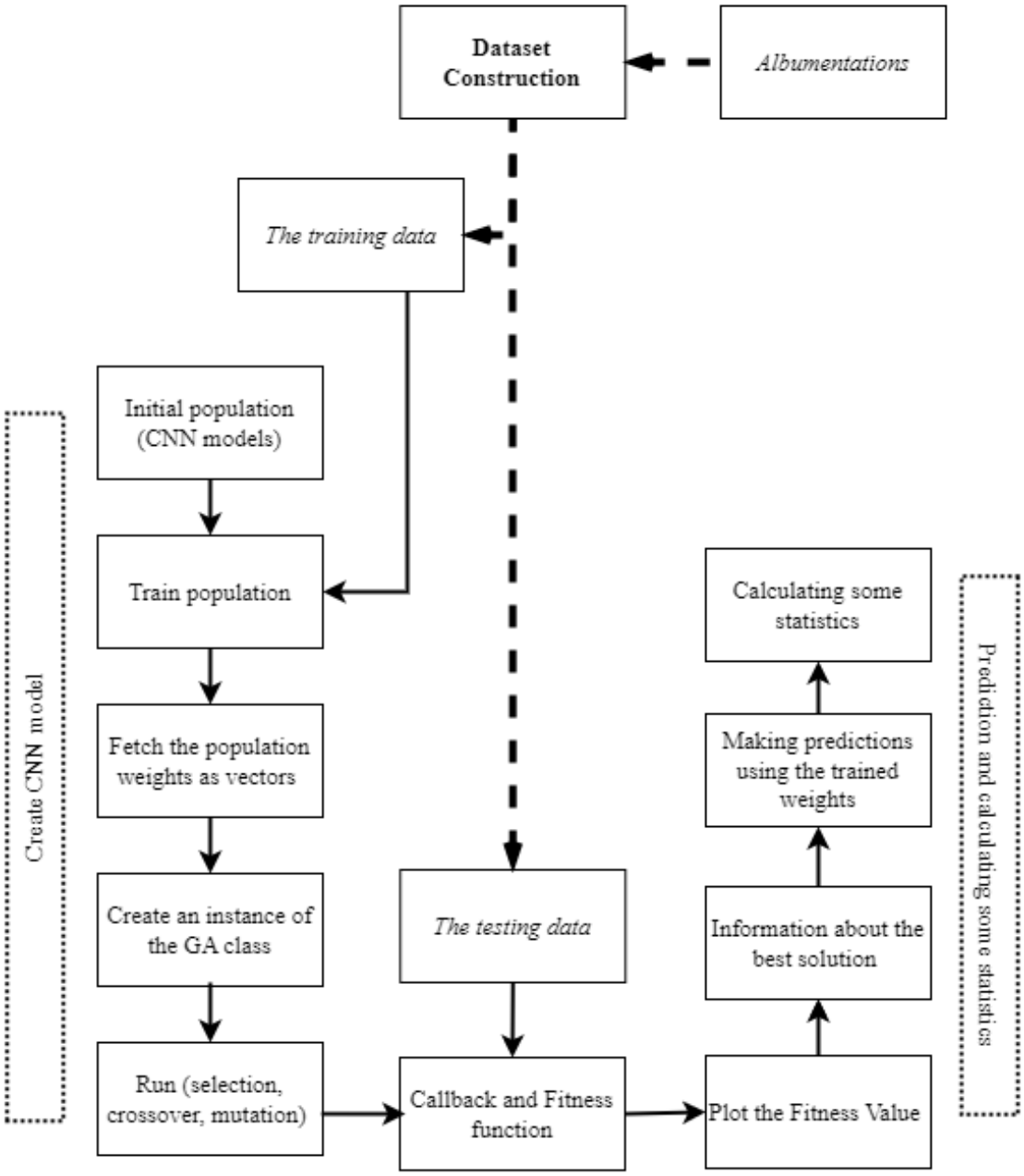


Figure 19: GACNN Framework

According to the reality of ceramics tiles production and the paper "Surface defects detection for ceramic tiles using image processing and morphological techniques" belong to Elbehiery, H., Hefnawy, A., & Elewa, M. ceramic tiles common defects are blob, eclipse, crack, edge, pinhole, scratch and defect free.



Figure 20: Common defect types of ceramic tiles surface

Table 2 below has a details description of these defects.

Table 2: Details description of ceramic's defect

Type of defect	Desciption
BLOB	is a defect, the spot located on the surface, which has a shape like a water drop but erratic, and the radius last from some mm to some cm.
SCRATCH	is a defect located on the surface, which is a missing part or multi-part in ceramic tiles with random shapes and made uneven surface, the size of the defect last from some mm to some cm.
ECLIPSE	is a defect that often appears on the surface, the discoloration loses the texture of the tile and tends to spread on the surface of the tile, with a size from a few mm to several tens of centimeters.
PINHOLE	is a defect with one or more circular shapes (spots, holes, pits) and black in color, usually located on the surface of the brick, with a radius of several mm.
EDGE	is a defect that loses single or multi-part, usually at the corner (edge) of the tile, with the size from a few cm to several tenths of cm.
CRACK	is a break, crack or cut in the surface of a brick, shaped as single or multi-long slots that divide the brick into two or more but not separate parts, ranging in size from a few millimeters to several tenths of cm.
DEFECT FREE	is when the surface of the tile is smooth (no cracks, pinholes, eclipses or blobs) and ensures the integrity of the tile (no edges, scratches).

The surface and faults of various products may call for various CNN structures, such as those with varied numbers of layers and kernels. It can be difficult to find the best CNN model for classifying flaws in products based on their surface. In this paper, a method based on evolutionary algorithms is suggested to automatically create the best CNN model. In particular, it is necessary to initially identify the general structure of CNN, including the number of convolutional layers, pooling layers, and fully connected layers. Then, depending on the designed GA, CNN structure parameters are tuned.

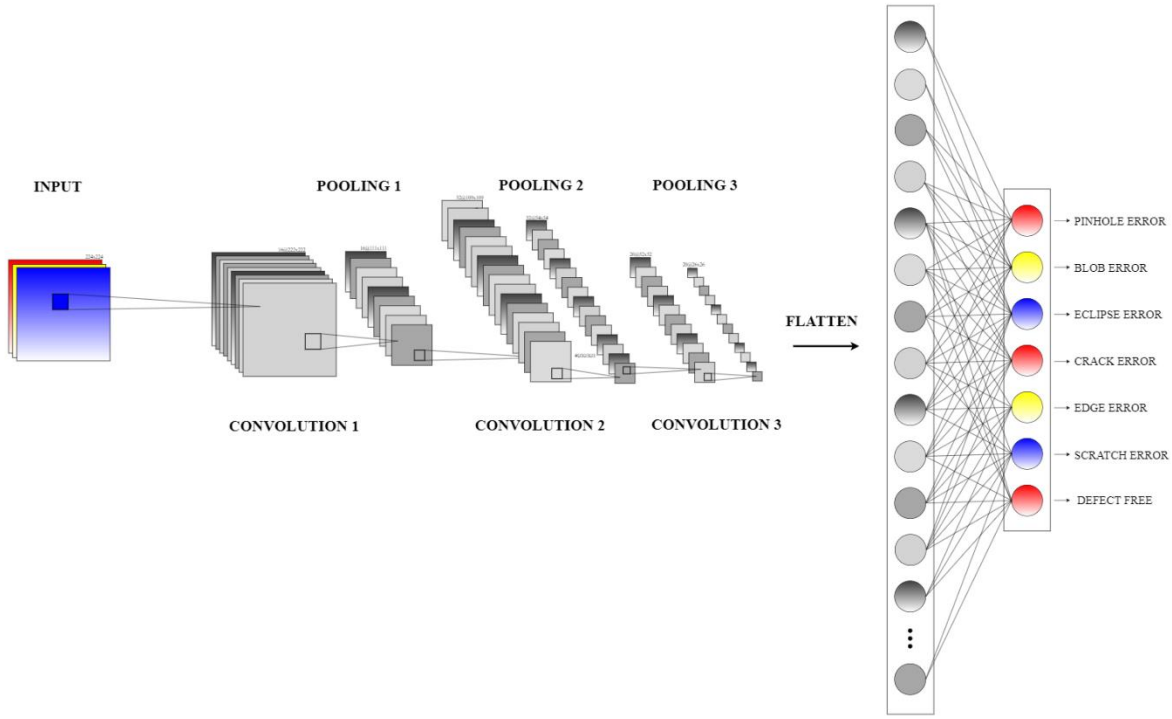


Figure 21: Example of fixed CNN structure

The first step is to create an initial population using an encoding routine. Every set of CNN structure parameters is represented by a chromosome. The processes of crossover and mutation are used to produce new offspring. Parents and children make up the transitory population that is utilized to assess and choose the next generation. Each chromosome's values in the test population are decoded and allocated to the CNN model. Based on a predetermined training dataset, CNN models are improved. The best CNN models are then assessed using the test dataset to produce fitness values for each chromosome (similar to the accuracy formula). A selection procedure is used to choose the chromosomes for the following generation based on fitness ratings. Up until the termination condition is satisfied, this process is iterated.

Choosing an activation function impacts significantly to capabilityand performance of the model. Table 3 below show a comparison of activation function.

Table 3: Details description of activation functions

NAME	ADVANTAGE	DISADVANTAGE	FORMULA
Sigmoid	Should be used to solve the binary classification Usually used in the last layer	Hard to converge The derivative is close to 0	$\sigma(x) = \frac{1}{1+e^{-x}}$
Tanh	Suitable when the average euqal to 0 (zero)	The derivative of x towards the two poles would approach 0 => learning rate slower	$\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
ReLU	Suitable when don't have any constrant with z	The derivative of x < 0 would approach 0 => learning rate slower	$f(x) = max(0, x)$
Leaky ReLU	Suitable when don't have any constrant with z	The derivative of x < 0 would approach α (α <<) => learning rate slower	f(x) = max(αx, x) với α << 1
eLu	Suitable when don't have any constrant with z	learning rate slower because of the exp () function	$\begin{cases} x & x \geq 0 \\ \alpha(e^{x}-1) & x < 0 \end{cases}$
Linear	Learning rate faster Range (-∞,+ ∞)	Backpropagation is not possible because the derivative always constant and it makes all the hidden ones 1 if linear is used as the activation function	f(x) =x

As seen in Figure 4, each parameter of a CNN structure is represented by a chromosome. With two parts, first one is feature extraction has: the convolution layer: num filters, kernel size, and activation function as the pooling layer has two types: AveragePooling and MaxPooling2D, but the same two gens are pool size and stride. Through the GA method, models for a few convolution layers and pooling layers would rise with each step. The last one is a classification that has the flatten layer with non-parameter and 2 dense layers with two gens: num_neurons and activation function. Especially, in dense layer 2, we usually have num_neurons equal to num_classes and the activation_function is softmax to have only one conclusion based on the sort.

Convolution 1			Pooling 1		Convolution 1			Pooling 1		Dense 1		Dense 2	
16	3	3	2	1	32	3	1	2	2	20	3	7	7

Convolution 1			Pooling 1		Convolution 1			Pooling 1		Convolution 1			Pooling 1		Dense 1		Dense 2	
16	3	3	2	1	32	3	1	2	2	30	4	5	2	2	60	3	7	7

Figure 22: Example of chromosome

For loop crossover, mutation, and selection, this chromosome would be introduced to a population. Currently, the library has a variety of selections (rank, random, tournament, roulette wheel, stochastic universal and customize selection), crossover (single point, two points, uniform and scattered) and mutation such as: random mutation, swap mutation, inversion mutation and scramble mutation.

Then, the best solution with the highest fitness—that weight and model—would be defined using the accuracy from the fitness function (with test image). The values mentioned above would be applied to the detection. The model compares the imported image or video with the features in the labels of the trained model after extracting the features from the video stream, and the outcome is shown in figure 21 below.



Figure 23: UI detection result by GACNN model

3.3 Operation Management Module - Beta

3.3.1 Aggregate Planning

This function focus on: Demand, Productivity, Labor Cost, Inventory Cost, and Relative Factors are taken into account from the sample dataset then using genetic algorithm to optimize the Aggregate Planning.

Table 4: Sample data of aggregate planning

ID	Demand	Production Cost	Holding Cost	Labor Cost	Overtime Cost	Labor Hour	Overtime Hour
1	100	7	3	15	22.5	120	30
2	100	8	4	15	22.5	130	40
3	150	8	4	18	27	120	40
4	200	8	4	18	27	150	30
5	150	7	3	15	22.5	100	30
6	100	8	2	15	22.5	100	30
…	…	…	…	…	…	…	…

After entering the cost, productivity, and demand data. To plan production with the lowest possible cost, we'll use the "Aggregate Production Planning" tool. Additionally, we'll utilize the "Create" or "Update" function if the value has to be added or altered. Once the "Current Data" portion has been verified to be accurate, we would choose "Predict" to obtain the optimal aggregate plan.

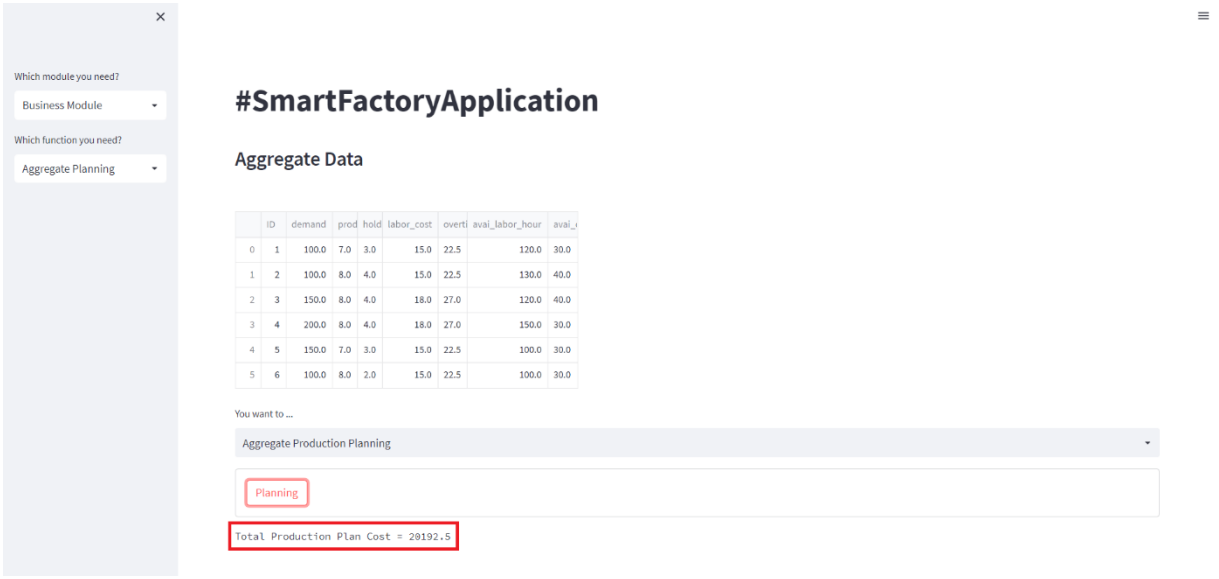


Figure 26: Aggregate Planning UI

3.3.2 Predictive Maintenance

By machine learning application, from the sample dataset using the Random Forest algorithm to consider Process Temp, Torque and Tool wear (Input) to predict Failure Type (Output) and evaluate the prediction and forecast.

Table 5: Sample data of predictive maintenance

UDI	Product ID	Air Temp (K)	Process Temp (K)	Speed (rpm)	Torque (Nm)	Tool Wear (min)	Failure Type
1	L47181	298.2	308.7	1408	46.3	3	No Failure
2	L47182	298.1	308.5	1498	49.4	5	No Failure
3	L47257	298.8	308.9	1455	41.3	208	Tool Failure
4	L47184	298.2	308.7	1408	40	9	No Failure
…	…	…	…	…	…	…	…
999	L47249	298.9	309	1410	65.7	191	Power Failure

Table 6: Benchmark of PM's models

No.	Models	Accuracy	Runtime(s)	Note
1	RandomForestClassifer	98.900	0.993097	I find that Random Forest has the highest accuracy with runtime acceptable when using fit() to compare the results of models.
2	StackingClassifer	98.867	1.784672	
3	BaggingClassifer	98.800	0.275748	
4	GradientBoostingClassifer	98.800	1.110636	
5	AdaBoostingClassifer	97.667	0.364328	
6	MPLClassifer	97.467	3.399050	
7	LogisticRegression	97.333	0.208842	
8	KNeughborsClassifer	96.067	0.1823395	

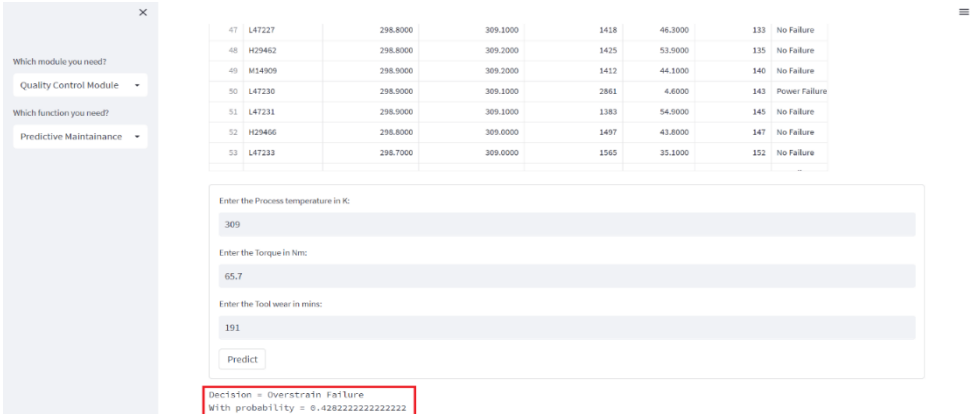


Figure 29: Predictive Maintenance UI

3.4 Business Module

3.4.1 Demand Forecasting

Fuel price is taken into account as an input when applying a model of linear regression using a sample data set to anticipate weekly sales (Output).

Table 7: Sample data of demand forecasting

Dept	Date	Weekly Sale	Is Holiday	Temperaturee	Fuel Price
1	2010-02-05	24924.5	False	42.31	2.572
2	2010-02-05	50605.27	False	41.81	2.572
3	2010-02-05	13740.12	False	40.20	2.572
4	2010-02-05	39954.04	True	40.35	2.572
5	2010-02-06	23333.38	False	35.15	2.572
6	2010-02-06	34740.20	False	32.11	2.572
7	2010-02-06	19154.04	True	35.17	2.572
8	2010-02-06	24213.33	False	35.15	2.594
9	2010-02-07	33381.11	False	35.15	2.594
…	…	…	…	…	…
999	2011-02-02	16930.99	False	35.02	2.875

We would forecast Weekly Sales based on Fuel Price once the linear regression model has been configured to train and predict.

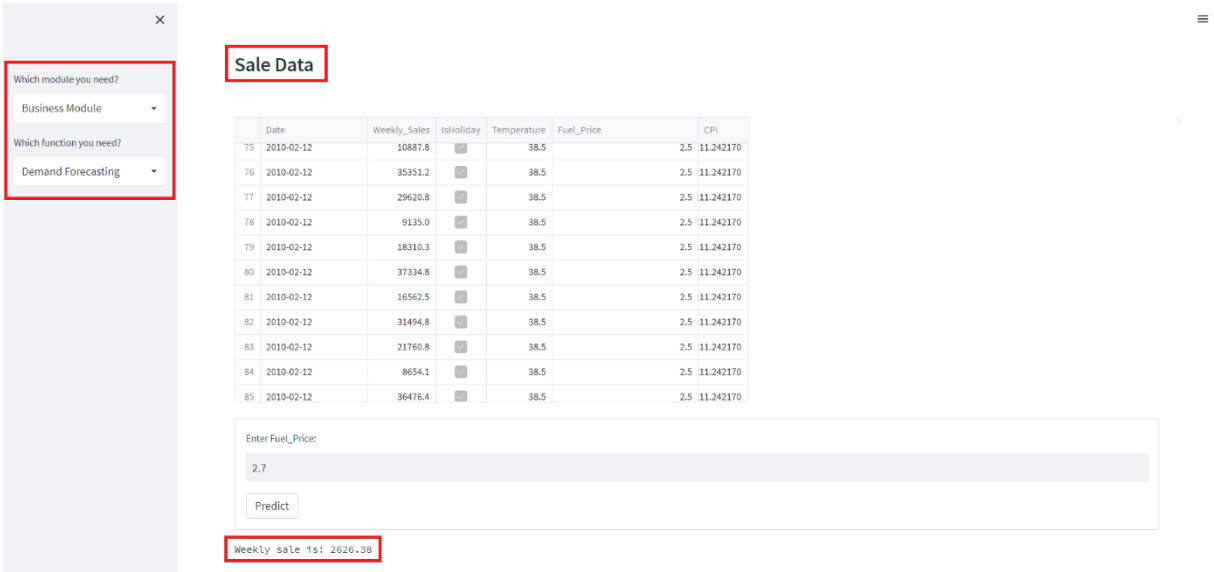


Figure 32: Demand Forecasting UI

3.4.2 Payroll

Due to the beta version, I recommend a straightforward concept, such as importing an excel file into the database to extract specific information about ID, name, number of hours worked, and number of overtime hours, and then calculating the rate per hour based on those numbers. Net pay is the salary an employee actually got after deducting all applicable taxes them saved in back to the database.

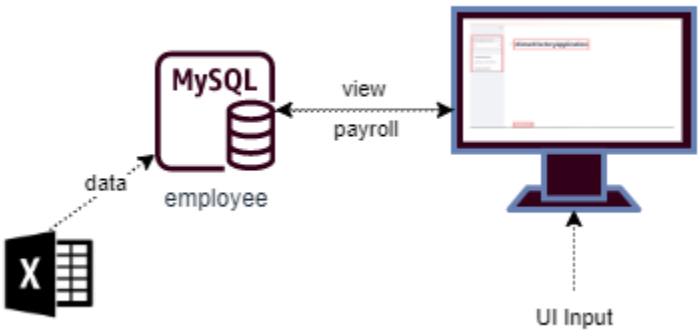


Figure 35: Payroll's architecture

In addition, the payroll value may be amended by the ID such for an extra bonus.

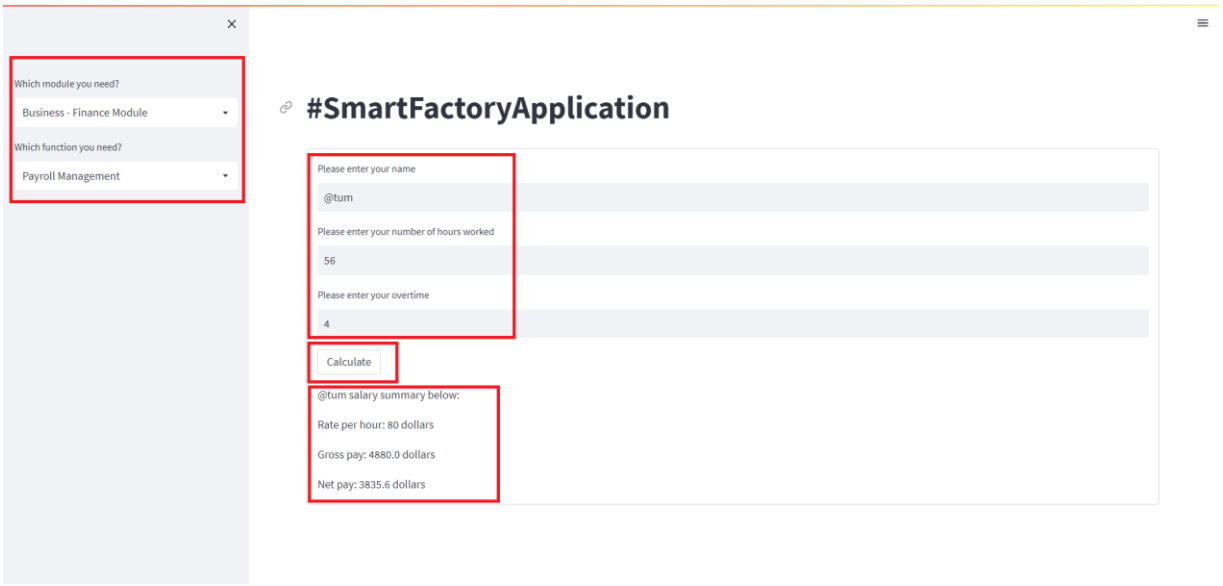


Figure 38: Payroll user interface

REFERENCES

[1]: Shariatzadeh, N., Lundholm, T., Lindberg, L., & Sivard, G. (2016). *Integration of digital factory with smart factory based on Internet of Things. Procedia Cirp, 50, 512-517.*

[2]: Rahman, R. A., Aziz, N. S. A., Kassim, M., & Yusof, M. I. (2017, April). *IoT-based personal health care monitoring device for diabetic patients*. In 2017 IEEE Symposium on Computer Applications & Industrial Electronics (ISCAIE) (pp. 168-173). IEEE.

[3]: Mazhar, M. S., Saleem, Y., Almogren, A., Arshad, J., Jaffery, M. H., Rehman, A. U., ... & Hamam, H. (2022). *Forensic Analysis on Internet of Things (IoT) Device Using Machine-to-Machine (M2M) Framework*. Electronics, 11(7), 1126.

[4]: Silvestre-Blanes, J., Albero Albero, T., Miralles, I., Pérez-Llorens, R., & Moreno, J. (2019). *A public fabric database for defect detection methods and results*. Autex Research Journal, 19(4), 363-374.

[5]: Shire, A. N., Khanapurkar, M. M., & Mundewadikar, R. S. (2011). *Plain Ceramic Tiles Surface Defect Detection Using Image Processing.* 2011 Fourth International Conference on Emerging Trends in Engineering & Technology.

[6]: Boukouvalas, C., Kittler, J., Marik, R., Mirmehdi, M., & Petrou, M. (1995). *Ceramic tile inspection for colour and structural defects*. Proceedings of AMPT95, 390-399.

[7]: Mantau, A. J., Widayat, I. W., Adhitya, Y., Prakosa, S. W., Leu, J. S., & Köppen, M. (2022, June). *A GA-Based Learning Strategy Applied to YOLOv5 for Human Object Detection in UAV Surveillance System*. In 2022 IEEE 17th International Conference on Control & Automation (ICCA) (pp. 9-14). IEEE.

[8]: Sun, Y., Xue, B., Zhang, M., Yen, G. G., & Lv, J. (2020). *Automatically designing CNN architectures using the genetic algorithm for image classification*. IEEE transactions on cybernetics, 50(9), 3840-3854.

[9]: Çınar, Z. M., Abdussalam Nuhu, A., Zeeshan, Q., Korhan, O., Asmael, M., & Safaei, B. (2020). *Machine learning in predictive maintenance towards sustainable smart manufacturing in industry 4.0. Sustainability*, 12(19), 8211.

[10]: Zeebaree, D. Q., Haron, H., & Abdulazeez, A. M. (2018, October). *Gene selection and classification of microarray data using convolutional neural network*. In 2018 International Conference on Advanced Science and Engineering (ICOASE) (pp. 145-150). IEEE.

APPENDIX

APPENDIX 1: GENERAL CODE OF ADUSIN

APPENDIX 2: DETAIL CODE OF THE DATA COLLECTION BY ESP8266

APPENDIX 3: DETAIL CODE OF CERAMIC DETECTION BY GACNN

APPENDIX 1: GENERAL CODE OF ADUSIN

```python
# IMPORT LIB
import streamlit as st
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import ...

# IMPORT FUNCTION
from module_db.process import *
from module_om.aggregate import *
from module_bf.linear_predict import *
from ...


# DATABASE
def loaded_df():
    machine_data = sqlalchemy.create_engine('mysql+pymysql://root:@localhost:3325/machine', pool_recycle=3600)
    machine_1_data = pd.read_sql_table("machine_1", machine_data)

    ...
    return machine_data, machine_1_data, ...

def get_detection_folder():...


# MAIN
if __name__ == '__main__':
    st.set_page_config(page_title='AdUsin by @tum',
                       page_icon='random', layout='wide',
                       initial_sidebar_state="expanded")
    st.title("#SmartFactoryApplication")
    option_main = st.sidebar.selectbox('Which module you need?', ('', "Database Module", "Operation Management Module", "Business - Finance Module", "Testing"))
    placeholder = st.empty()

    # ------------------------------------------------------------------- #

    # DATABASE MODULE
    if option_main == "Database Module":
        option_machine = st.sidebar.selectbox('Which database you need?',('', "Machine 1", "Machine 2"))
        st.write(f"Table below shows the summary data of {option_machine}")
        if option_machine == "Machine 1":
            # Dashboard
            hide_dataframe_row_index = """..."""

            # Inject CSS with Markdown
            st_autorefresh(interval=5 * 1000, key="loading")
            st.markdown(hide_dataframe_row_index, unsafe_allow_html=True)
            table = st.dataframe(machine_1_data.style.format(subset=['humidity', 'temperature', 'pressure', 'torque'],formatter="{:.1f}"))
            st_autorefresh(interval=10 * 1000, key="loading machine 1 data")


```

```python
            # CRUD
            option_crud = st.sidebar.selectbox('You want to ...',('', "Create", "Update", "Delete"))
            if option_crud == "Create":
                with st.form(key="Create"):
                    humidity = st.text_input('Enter current Humidity', 'Type here !!!')
                    temperature = st.text_input('Enter current Temperature', 'Type here !!!')
                    pressure = st.text_input('Enter current Pressure', 'Type here !!!')
                    torque = st.text_input('Enter current Torque', 'Type here !!!')
                    create_button = st.form_submit_button("Create")
                # CONFIG
                if create_button:
                    new_data = add_data(humidity, temperature, pressure, torque)
                    table.dataframe(get_all_data().style.format(subset=['humidity', 'temperature', 'pressure', 'torque'],formatter="{:.1f}"))

            elif option_crud == "Update":...

            elif option_crud == "Delete":...

        elif option_machine == "Machine 2":
            pass

     # ------------------------------------------------------------------- #

    # OPERATION MANAGEMENT MODULE
     elif option_main == "Operation Management Module":
            option_om = st.sidebar.selectbox( 'Which function you need?',('', "Aggregate Planning", "Predictive Maintainance", "Quality Control"))
            # This code block for Aggregate Planning
            if option_om == "Aggregate Planning":
                    # Dashboard
             hide_dataframe_row_index = """..."""
             # Inject CSS with Markdown
             st.markdown(hide_dataframe_row_index, unsafe_allow_html=True)
             table = st.dataframe(aggregate_data_short.style.format(subset=["demand", "production_cost", "holding_cost", "labor_cost","overtime_cost", "avai_labor_hour", "avai_over_hour"])
             # CRUD
             option_crud = st.selectbox('You want to ...',('', "Create", "Update", "Aggregate Production Planning"))
             if option_crud == "Create":...

             elif option_crud == "Update":...

             elif option_crud == "Delete":...

            # This code block for Predictive Maintainance
        elif option_om == "Predictive Maintainance":...

        # This code block for Quality Control
        elif option_om == "Quality Control":
            option_qc = st.sidebar.selectbox('Which function you need?',('', "Control Chart", "Monitoring", "Material Defect Detection"))

            # This code block for Control chart
            if option_qc == "Control Chart":
                option_machine = st.sidebar.selectbox('Please choose machine for Control Chart',('', "Machine 1", "Machine 2"))
                if option_machine == "Machine 1":
                    # RUN Control Chart Machine 1
                    subprocess.run([f"{sys.executable}", "control_chart.py"])
                elif option_machine == "Machine 2":...
```

```
        # This code block for Monitoring
        elif option_qc == "Monitoring":
            # RUN TRACKING Machine 1
            subprocess.run([f"{sys.executable}", "monitor.py"])
            # LED Machine 1 Status
            m1_current_value = m1_status_data["Stat"][0]

        # This code block for Material Defect Detection
        elif option_qc == "Material Defect Detection":
            option_detect = st.sidebar.selectbox('Please choose material to detection',('', "Ceramic GACNN", "Ceramic Ver 1"))

            # GACNN ALGORITHM
            if option_detect == "Ceramic GACNN":
                source = ("", "Image", "Auto Detection", "Video", "Real-time")
                source_index = st.sidebar.selectbox("Input", range(len(source)), format_func=lambda x: source[x])

                #  IMAGE
                if source_index == 1:
                    uploaded_file = st.sidebar.file_uploader(
                        "Upload Image", type=['png', 'jpeg', 'jpg'])
                    if uploaded_file is not None:
                        is_valid = True
                        with st.spinner(text='Loading...'):
                            st.image(uploaded_file, width=400)
                            picture = Image.open(uploaded_file)
                            picture.save(f'datast/cera_upload/{uploaded_file.name}')
                            if st.button('Detection'):
                                gacnn_detection = GACNN_DETECTION()
                                result = gacnn_detection.predict_single(f'datast/cera_upload/{uploaded_file.name}')
                                st.write(f"The defect of img is: {result[1]} with code [{result[0]}]")
                # AUTO DETECTION
                elif source_index == 2:...

                # VIDEO
                elif source_index == 3:...

                # REALTIME
                elif source_index == 4:...

            # YOLO V7
            elif option_detect == "Fabric Yolo":
                source = ("Image", "Video", "Real-time")
                source_index = st.sidebar.selectbox("Input", range(len(source)), format_func=lambda x: source[x])

                # IMAGE
                if source_index == 1:...

                # VIDEO
                elif source_index == 2:...

                # REALTIME
                elif source_index == 3:...
```

```
163
164     # ---------------------------------------------------------------------- #
165
166     # BUSINESS - FINANCE MODULE
167     elif option_main == "Business - Finance Module":
168         option_busifin = st.sidebar.selectbox('Which function you need?',('', "Demand Forecasting", "Payroll Management"))
169
170         # This code block for Demand Forecasting
171         if option_busifin == "Demand Forecasting":
172             st.subheader("Sale Data")
173             hide_dataframe_row_index = """..."""
174             # Inject CSS with Markdown
175             st.markdown(hide_dataframe_row_index, unsafe_allow_html=True)
176             table = st.dataframe(sale_data_short.iloc[:100].style.format(subset=["Weekly_Sales", "Temperature", "Fuel_Price"],formatter="{:.1f}"))
177             with st.form(key="Predict_Demand"):
178                 fuel_index = st.text_input("Enter Fuel_Price: ")
179                 predict_button = st.form_submit_button("Predict")
180             # CONFIG
181             if predict_button:
182                 sale_predict = predict(float(fuel_index), weight, bias)
183                 st.text(f"Weekly sale is: {round(sale_predict, 2)}")
184
185         # This code block for Payroll
186         elif option_busifin == "Payroll Management":...
187
188     # ---------------------------------------------------------------------- #
189
190     # TEST MODULE
191     elif option_main == "Testing":
```

APPENDIX 2: DETAIL CODE OF THE DATA COLLECTION BY ESP8266

```
# include <ESP8266HTTPClient.h>
# include <ESP8266WiFi.h>
# include <WiFiClient.h>
# include <DHT.h>

# define LED_D8 D8 //--> Defines an LED Pin. D8 = GPIO15

float temperature;
float humidity;
float pressure;
float torque;

char ssid[] = "LoanDien"; // ten Wifi
char pass[] = "26122003"; // pass Wifi

HTTPClient http;
WiFiClient client1;
DHT dht(D1, DHT11);

unsigned long last = 0;

void ConnectWiFi();
void dataSensor();
void Send_Data_PHP(String temperature, String humidity, String torque, String pressure);
void Get_Data_PHP();
void setup()
{
    Serial.begin(9600);
    pinMode(LED_D8, OUTPUT); // --> On Board LED port Direction output
    digitalWrite(LED_D8, LOW); // --> Turn off Led On Board
    ConnectWiFi();
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());
    dht.begin();

    digitalWrite(LED_D8, HIGH); // turn the LED on(HIGH is the voltage level)
    delay(500); // wait for a second
    digitalWrite(LED_D8, LOW); // turn the LED off by making the voltage LOW
    delay(500); // wait for a second

    digitalWrite(LED_D8, HIGH); // turn the LED on (HIGH is the voltage level)
    delay(500); // wait for a second
    digitalWrite(LED_D8, LOW); // turn the LED off by making the voltage LOW
    delay(500); // wait for a second


    last = millis();
```

```
}
// ------------------------------------------------------------------------------

/ *Infinite Loop * /
void loop()
{
    if (WiFi.status() == WL_CONNECTED) // kierm tra wifi con ket noi k
    {
        if (millis() - last > 500) // 500ms = 0.5s // cu moi 0.5s update 1 lan
        {
            dataSensor();
            Send_Data_PHP(String(temperature), String(humidity), String(torque), String(pressure));
            Get_Data_PHP();
            last = millis();
        }
    }
    else
    {
        Serial.println("Wifi disconected\n");
        delay(2000);
    }
}

void ConnectWiFi()
{
    WiFi.begin(ssid, pass);

    while (WiFi.status() != WL_CONNECTED)
    {
        delay(500);
        Serial.print(".");
    }
    Serial.println("Connect WIFI!!!");
}

void dataSensor()
{
    humidity = dht.readHumidity();
    temperature = dht.readTemperature();
    torque = (random(40, 50)) / 1.0;
    pressure = (random(20, 25)) / 1.0;
}
```

```
98
99  void Send_Data_PHP(String temperature, String humidity, String torque, String pressure)
100 {
101     String
102     c1 = "http://192.168.55.47/demosensor/machine_1.php?temperature=";
103     c1 += String(temperature);
104
105     String
106     c3 = "&humidity=";
107     c1 += String(c3);
108     c1 += String(humidity);
109
110     String
111     c5 = "&pressure=";
112     c1 += String(c5);
113     c1 += String(pressure);
114
115     String
116     c7 = "&torque=";
117     c1 += String(c7);
118     c1 += String(torque);
119     http.begin(c1); // params có webClient Client
120     http.addHeader("Content-Type", "application/x-www-form-urlencoded");
121     String httpRequestData = "temperature=" + String(temperature) + "&humidity=" + String(humidity) + "&pressure=" + String(pressure) + "&torque=" + String(torque) + "";
122     // Serial.println(httpRequestData);
123     Serial.println("----------");
124     int httpResponseCode = http.POST(httpRequestData);
125     // Serial.println(httpResponseCode);
126
127     if (httpResponseCode == 200)
128     {
129         Serial.print("Send Data Okelala .,.");
130         Serial.println(httpRequestData);
131
132         delay(200);
133     }
134     else
135     {
136         Serial.println("Can't send data to database!!!");
137         Serial.println(httpRequestData);
138     }
139     http.end();
140     //=============================================================================================================
141 }
142
143
144
145
146
147
```

```
148
149 void Get_Data_PHP()
150     {
151     // main code here, to run repeatedly:
152
153     // ----------------------------------------Getting Data from MySQL Database
154     String LinkGet, getData;
155     int id = 0; // --> ID in Database
156
157     LinkGet = "http://192.168.55.47/demosensor/machine_1_status_esp.php"; // --> Make a Specify request destination
158     getData = "ID=" + String(id);
159     Serial.println("----------------Connect to Server----------------");
160     Serial.println("Get LED Status from Server or Database");
161     Serial.print("Request Link : ");
162     Serial.println(LinkGet);
163     http.begin(LinkGet); // --> Specify request destination
164     http.addHeader("Content-Type", "application/x-www-form-urlencoded"); // Specify content - type header
165     int httpCodeGet = http.POST(getData);       // --> Send the request
166     String payloadGet = http.getString();       // --> Get the response payload from server
167     Serial.print("Response Code : ");           // --> If Response Code = 200 means Successful connection, if -1 means connection failed.
168     Serial.println(httpCodeGet);                // --> Print HTTP return code
169     Serial.print("Returned data from Server : ");
170     Serial.println(payloadGet);                 // --> Print request response payload
171     // Serial.println(strlen(String(payloadGet)));
172     if (payloadGet == "1\r\n")
173         {
174             digitalWrite(LED_D8, HIGH); // --> Turn on Led
175             Serial.println("Turn on Led");
176         }
177         if (payloadGet == "2\r\n")
178             {
179                 digitalWrite(LED_D8, LOW); // --> Turn off Led
180                 Serial.println("Turn off Led");
181             }
182
183         // ----------------------------------------
184
185         Serial.println("----------------Closing Connection----------------");
186         http.end();         // --> Close
187         connection
188         // Serial.println();
189         // Serial.println("Please wait 5 seconds for the next connection.");
190         // Serial.println();
191         // delay(5000);     // --> GET Data at every 5 seconds
192 }
```

APPENDIX 3: DETAIL CODE OF CERAMIC DETECTION BY GACNN

```
import numpy
import pygad.cnn
import pygad.gacnn
import pygad
import ...

def fitness_func(solution, sol_idx):
    global GACNN_instance, data_inputs, data_outputs

    predictions = GACNN_instance.population_networks[sol_idx].predict(data_inputs=data_inputs)
    correct_predictions = numpy.where(predictions == data_outputs)[0].size
    solution_fitness = (correct_predictions/data_outputs.size)*100

    return solution_fitness

def callback_generation(ga_instance):
    global GACNN_instance, last_fitnessm, start_time

    population_matrices = pygad.gacnn.population_as_matrices(population_networks=GACNN_instance.population_networks,population_vectors=ga_instance.population)
    GACNN_instance.update_population_trained_weights(population_trained_weights=population_matrices)

    print("Generation = {generation}".format(generation=ga_instance.generations_completed))
    print("Population: ", ga_instance.population)
    print("The best solution: ", ga_instance.best_solutions)
    print("Fitness    = {fitness}".format(fitness=ga_instance.best_solutions_fitness))
    print("Solutions Fitness: ", ga_instance.solutions_fitness)
    print("--------------------------------------------------------------------------------")
    # Predicting the outputs of the data using the best solution.
    solution, solution_fitness, solution_idx = ga_instance.best_solution()
    predictions = GACNN_instance.population_networks[solution_idx].predict(data_inputs=data_inputs_test)
    print("Predictions of the trained network : {predictions}".format(predictions=predictions))
    print("The resullt : {data_outputs}".format(data_outputs=data_outputs_test))
    print("--------------------------------------------------------------------------------")
    num_wrong = numpy.where(predictions != data_outputs_test)[0]
    num_correct = data_outputs_test.size - num_wrong.size
    accuracy = 100 * (num_correct/data_outputs_test.size)
    print("Number of correct classifications : {num_correct}.".format(num_correct=num_correct))
    print("Number of wrong classifications : {num_wrong}.".format(num_wrong=num_wrong.size))
    print("Classification accuracy : {accuracy}.".format(accuracy=accuracy))
    print("--- %s seconds ---" % (time.time() - start_time))
    print("--------------------------------------------------------------------------------")


```

```
44
45  # Start to save terminal output
46  sys.stdout = open("record_fixed_cnn.txt", "w")
47
48  # Initial population
49  start_time = time.time()
50  dataset = numpy.load('dataset_ceramic_train.npz')
51  data_inputs = dataset["x"]
52  data_outputs = dataset["y"]
53
54  dataset_test = numpy.load('dataset_ceramic_test.npz')
55  data_inputs_test = dataset_test["x"]
56  data_outputs_test = dataset_test["y"]
57
58  sample_shape = data_inputs.shape[1:]
59  num_classes = 7
60
61  # Ran_list
62  conv_const = 16
63  ac_fun = {"sigmoid": 1, "tanh": 2, "relu": 3, "leakyrelu": 4, "elu": 5, "linearres": 6}
64  conv_acfun = random.choice(list(ac_fun.items()))
65  # print(conv_acfun[0], conv_acfun[1])
66
67  pool_poolsz = random.randint(2, 2)
68  pool_stride = random.randint(2, 2)
69
70  dense_acfun = ['relu', 'sigmoid', 'softmax']
71
72  input_layer = pygad.cnn.Input2D(input_shape=sample_shape)
73  pool_layer = None
74  struc_cnn = {}
75  gene_space_sequence = []
76  param_num = []
77  # Filter list control
78  filters_list = [0, 16, 64, 30, 20]
79  for i in range(1, random.randint(3, 4)):
80      if pool_layer is None:
81          conv_layer = pygad.cnn.Conv2D(num_filters=filters_list[i],
82                                        kernel_size=3,
83                                        previous_layer=input_layer,
84                                        activation_function=conv_acfun[0])
85          pool_layer = pygad.cnn.AveragePooling2D(pool_size=pool_poolsz,
86                                                  previous_layer=conv_layer,
87                                                  stride=pool_stride)
88
89
90
```

```
91
92
93      else:
94          conv_layer = pygad.cnn.Conv2D(num_filters=filters_list[i],
95                                        kernel_size=3,
96                                        previous_layer=pool_layer,
97                                        activation_function=conv_acfun[0])
98          pool_layer = pygad.cnn.MaxPooling2D(pool_size=pool_poolsz,
99                                              previous_layer=conv_layer,
100                                             stride=pool_stride)
101     # print(f"conv_layer_{i}: ", conv_const)
102     # Control chromosome
103     struc_cnn.update({f"conv_layer_{i}": {"num_filters": filters_list[i],
104                                           "kernel_size": 3,
105                                           "activation_function": f"{conv_acfun[0]}"},
106                       f"pool_layer_{i}": {"pool_size": pool_poolsz,
107                                           "stride": f"{pool_stride}"}
108                       })
109
110     gene_space_sequence.extend([filters_list[i], 3, conv_acfun[1]],)
111     gene_space_sequence.extend([pool_poolsz, pool_stride])
112     param_num.extend(conv_layer.layer_input_size)
113     param_num.extend(conv_layer.layer_output_size)
114     param_num.extend(pool_layer.layer_input_size)
115     param_num.extend(pool_layer.layer_output_size)
116     conv_const = conv_const * 2
117     conv_acfun = random.choice(list(ac_fun.items()))
118
119 flatten_layer = pygad.cnn.Flatten(previous_layer=pool_layer)
120
121 dense_layer_1 = pygad.cnn.Dense(num_neurons=4,
122                                 previous_layer=flatten_layer,
123                                 activation_function="relu")
124
125 dense_layer_2 = pygad.cnn.Dense(num_neurons=num_classes,
126                                 previous_layer=dense_layer_1,
127                                 activation_function="softmax")
128 # Last update chromosome
129 struc_cnn.update({f"dense_layer_1": {"num_neurons": 4,
130                                      "activation_function": "relu"},
131                   f"dense_layer_2": {"num_neurons": num_classes,
132                                      "activation_function": "softmax"}
133                   })
134
135
136
137
```

```
138
139 gene_space_sequence.extend([4, 3])
140 gene_space_sequence.extend([7, 7])
141
142 model = pygad.cnn.Model(last_layer=dense_layer_2,
143                         epochs=10,
144                         learning_rate=0.01)
145
146 model.summary()
147
148 cnn_vectors = pygad.cnn.layers_weights_as_vector(model=model)
149 print("cnn vector", cnn_vectors)
150 print("len param", len(cnn_vectors))
151 print("param num", param_num)
152
153 print("Structure of CNN :", struc_cnn)
154 print("-------------------------------------------------------------------------------------")
155 print("Gene space :", gene_space_sequence)
156 print("Len of Gene space :", len(gene_space_sequence))
157
158 # save gene_space to file
159 file = open("cnn_truc_fixed_cnn.txt", "w")
160 str_cnn_struc = repr(gene_space_sequence)
161 file.write(str_cnn_struc)
162 file.close()
163 print("-------------------------------------------------------------------------------------")
164
165 GACNN_instance = pygad.gacnn.GACNN(model=model,num_solutions=4)
166 print("GACNN_instance: ", GACNN_instance)
167
168 population_vectors = pygad.gacnn.population_as_vectors(population_networks=GACNN_instance.population_networks)
169 population_matrices = pygad.gacnn.population_as_matrices(population_networks=GACNN_instance.population_networks, population_vectors=population_vectors)
170
171 # GACNN params below !!!
172 initial_population = population_vectors.copy()
173 # Number of solutions to be selected as parents in the mating pool.
174 num_parents_mating = 2
175 num_generations = 10  # Number of generations.
176 parent_selection_choice = ['sss', 'rws', 'sus', 'random', 'tournament', 'rank']
177 parent_selection_type = "sss"  # Type of parent selection.
178 keep_parents = -1 # Number of parents to keep in the next population. -1 means keep all parents and 0 means keep nothing.
179 crossover_choice = ['single_point', 'two_points', 'uniform', 'scattered']
180 crossover_type = "single_point"  # Type of the crossover operator.
181 mutation_percent_genes = 10 # Percentage of genes to mutate. This parameter has no action if the parameter mutation_num_genes exists.
182 mutation_choice = ['swap', 'scramble', 'inversion', 'random', 'adaptive']
183 mutation_type = "random"  # Type of the mutation operator.
184
```

```
185
186 ga_instance = pygad.GA(num_generations=num_generations,
187                         num_parents_mating=num_parents_mating,
188                         initial_population=initial_population,
189                         fitness_func=fitness_func,
190                         parent_selection_type=parent_selection_type,
191                         keep_parents=keep_parents,
192                         crossover_type=crossover_type,
193                         mutation_percent_genes=mutation_percent_genes,
194                         mutation_type=mutation_type,
195                         save_best_solutions=True,
196                         on_generation=callback_generation)
197 ga_instance.run()
198 # After the generations complete, some plots are showed that summarize how the outputs/fitness values evolve over generations.
199 ga_instance.plot_fitness()
200
201 # Returning the details of the best solution.
202 solution, solution_fitness, solution_idx = ga_instance.best_solution()
203 print("------------------------------------------------------------------------------")
204 print("Parameters of the best solution : {solution}".format(solution=solution))
205 print(f"Len of the best original solution : {len(solution)}")
206 print("Fitness value of the best solution = {solution_fitness}".format(solution_fitness=solution_fitness))
207 print("Index of the best solution : {solution_idx}".format(solution_idx=solution_idx))
208 cnn_solution_final_metrics = pygad.cnn.layers_weights_as_matrix(model=model, vector_weights=solution)
209 print("cnn metrics", cnn_solution_final_metrics)
210 if ga_instance.best_solution_generation != -1:
211     print("Best fitness value reached after {best_solution_generation} generations.".format(best_solution_generation=ga_instance.best_solution_generation))
212
213 # Predicting the outputs of the data using the best solution.
214 predictions = GACNN_instance.population_networks[solution_idx].predict(data_inputs=data_inputs_test)
215 print("GACNN_instance: ", GACNN_instance)
216 print("GACNN_instance population network: ",GACNN_instance.population_networks[solution_idx])
217 print("Predictions of the trained network (test) : {predictions}".format(predictions=predictions))
218
219 # Calculating some statistics
220 num_wrong = numpy.where(predictions != data_outputs_test)[0]
221 num_correct = data_outputs_test.size - num_wrong.size
222 accuracy = 100 * (num_correct/data_outputs.size)
223 print("Number of correct classifications : {num_correct}.".format(num_correct=num_correct))
224 print("Number of wrong classifications : {num_wrong}.".format(num_wrong=num_wrong.size))
225 print("Classification accuracy : {accuracy}.".format(accuracy=accuracy))
226
227 # Save result
228 filename = 'result_to_day_fixed_cnn'
229 ga_instance.save(filename=filename)
```

THE END.

Thank you for taking your valuable time to review this documentation.
I'm really appreciated your feedback and advices.